Análise de Série Temporal (Número de Servidores da Secretária de Educação) - Prefeitura de Uberlândia

Drianny Jeronima da Silva Mamedio

2026-03-20

Introdução: Neste trabalho, estudou-se o comportamento do número de servidores da Secretaria de Educação do município de Uberlândia (MG), compreendendo seus padrões e variações com o objetivo de realizar predições para os próximos 12 meses. Este estudo possui relevância para o planejamento de políticas públicas e a gestão eficiente de recursos no sistema educacional municipal. Os dados foram extraídos do Catálogo de Dados Abertos da Prefeitura de Uberlândia, abrangendo o período de 2021 a 2025.

A metodologia estatística de series temporais é fundamental em modelagem de dados e análises preditivas contribuindo para tomada de decisões baseadas em evidências, estudos de comportamentos e padrões, sendo amplamente utilizada em diversas áreas das ciências. A partir da decomposição da série, confirmou-se que a variável apresenta uma tendência de crescimento ao longo dos anos e uma forte componente sazonal. Esta tendência indica um comportamento cíclico de contratações e desligamentos, com quedas acentuadas em janeiro, refletindo a dinâmica contratual da Secretaria de Educação

Metodologia: Para este trabalho foi utilizado o software R realizando o estudo da série através do seguinte fluxo de análise:

- Análise Exploratória: Identificação de tendência, sazonalidade, pontos discrepantes.
- Teste de Estacionariedade e Diferenciação.
- Modelagem: Testes com modelos paramétricos (ARIMA/SARIMA) e não paramétricos (GAM e STL).
- Validação: Uso de treino e teste para escolha do modelo com menor erro percentual absoluto médio (MAPE).
- Predição: Estimativa de previsão para 12 meses, utilizando um intervalo de confiança de 95%.

Após os testes e validações o melhor modelo escolhido para série foi STL+Naive o que indica que o quadro de contratação dos servidores da Secretária de Educação em Uberlândia possui um comportamento repetitivo e conservador. O modelo baseia-se na decomposição aditiva expressa como:

$$Y_t = T_t + S_t + R_t$$

Onde Y_t é o número total de servidores no tempo t. T_t representa o ciclo da séria. S_t são as oscilações periódicas. R_t representa o componente aleatório não explicado pelo modelo (ruído).

A fórmula da previsão para o horizonte h é dada por:

$$\hat{y}_{T+h} = y_T + h\left(\frac{y_T - y_1}{T-1}\right)$$

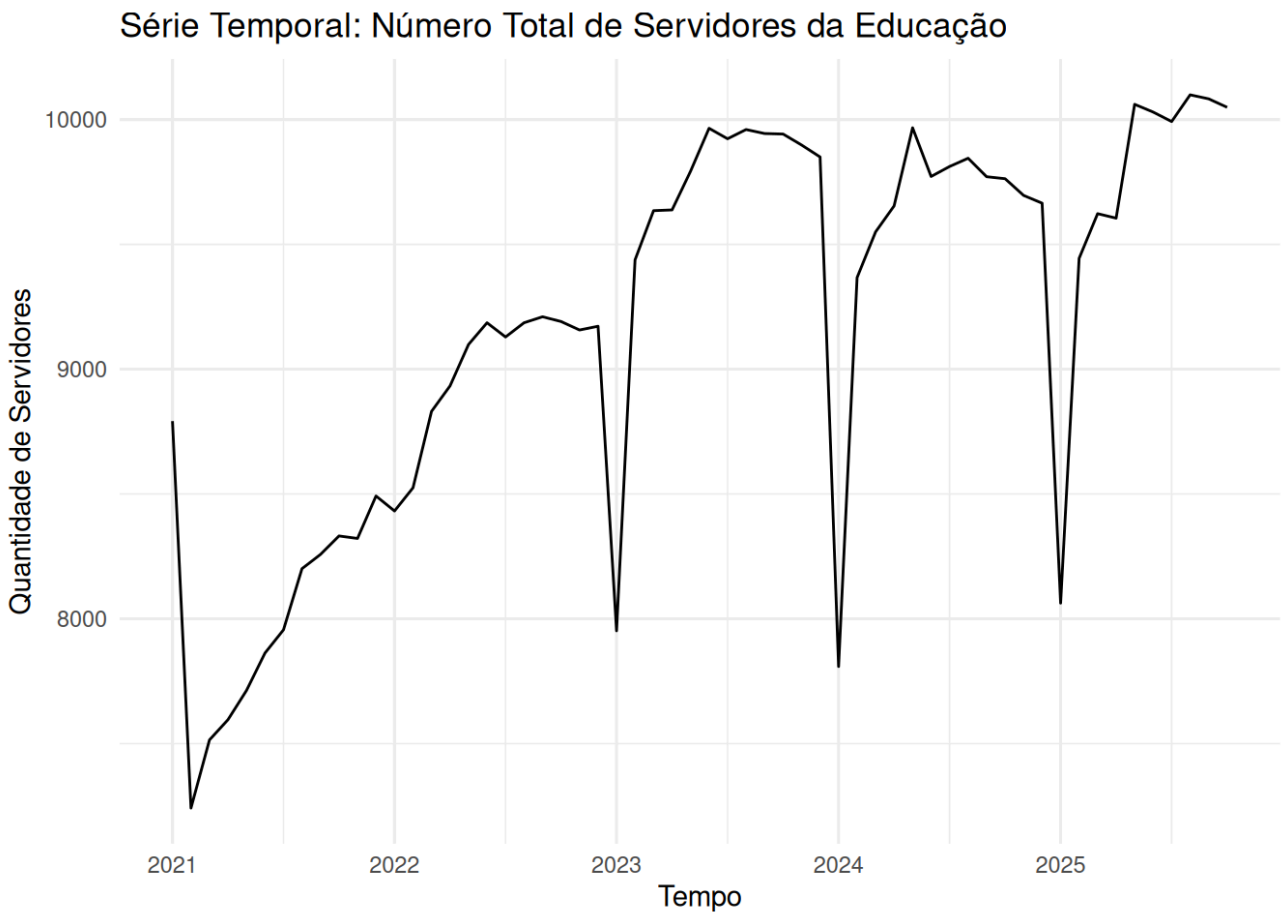
O termo $\left(\frac{y_T - y_1}{T-1}\right)$ representa a inclinação média da série, permitindo projetar um crescimento ou decréscimo linear baseado no comportamento capturado.

Análises:

```
knitr::opts_chunk$set(echo = TRUE)

# Carregando os pacotes
library(forecast)
library(tseries)
library(ggplot2)
library(readr)
library(scales)
library(tsibble)
library(feasts)
library(gridExtra)
library(zoo)
library(mgcv)

# Importando os dados
dados <- read.csv("dados_prefeitura_limpos_final.csv")

#garantindo que a coluna de data será lida corretamente
dados$data_ref <- as.Date(dados$data_ref)
dados <- dados[order(dados$data_ref), ]

#SELECIONANDO VARIÁVEL
serie_ts <- ts(dados$Educa.Número.de.Servidores.Total,
                     start = c(2021, 1),
                     frequency = 12)

#Análise exploratória

p1 <- autoplot(serie_ts) +
  ggtitle("Série Temporal: Número Total de Servidores da Educação") +
  xlab("Tempo") +
  ylab("Quantidade de Servidores") +
  theme_minimal()

cat("Serie Original\n")
```

```
## Serie Original
```

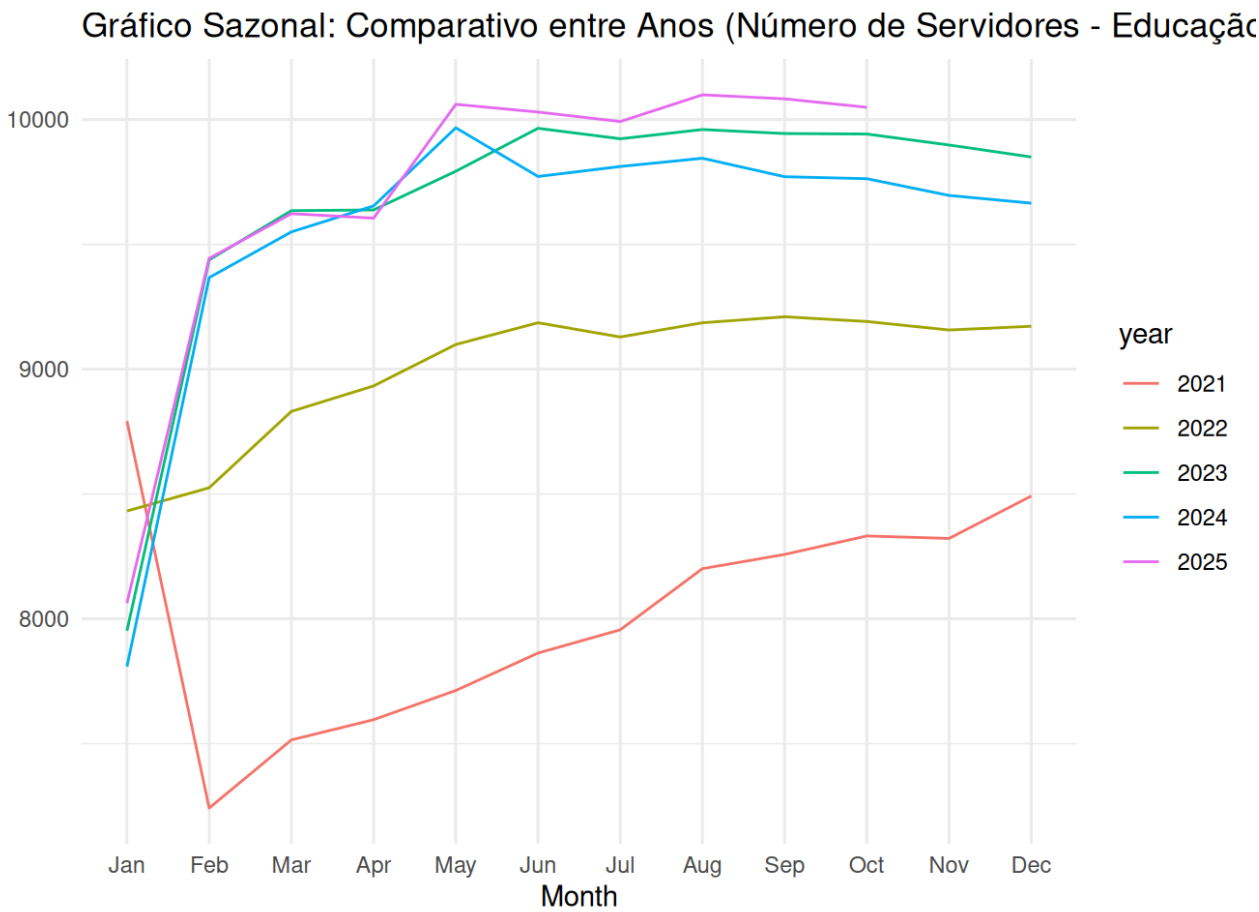
```
p1
```




```
p2 <- ggseasonplot(serie_ts) +
  ggtitle("Gráfico Sazonal: Comparativo entre Anos (Número de Servidores - Educação)")
+
  theme_minimal()

cat("Gráfico de sazonalidade\n")
```

```
## Gráfico de sazonalidade
```

```
p2
```



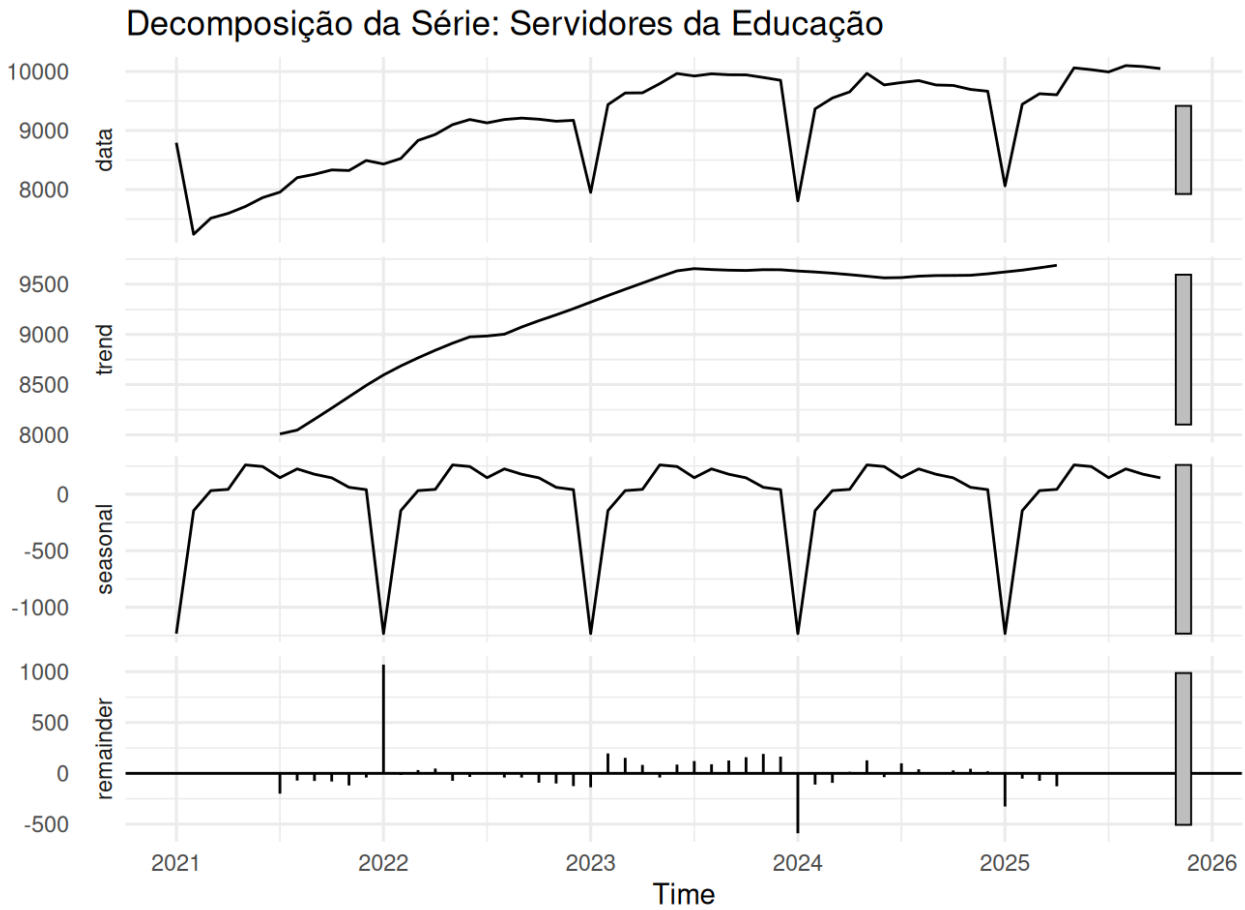

```
cat("A maioria das linhas dos gráficos iniciam com crescimento de Janeiro para Fevereir
o, porém o ano de 2021 se mostrou um ano atípico iniciando em queda.")
```

```
## A maioria das linhas dos gráficos iniciam com crescimento de Janeiro para Fevereiro,
porém o ano de 2021 se mostrou um ano atípico iniciando em queda.
```

```
p3 <- autoplot(decompose(serie_ts)) +
  ggtitle("Decomposição da Série: Servidores da Educação") +
  theme_minimal()

cat("Decomposição da série\n")
```

```
## Decomposição da série
```

```
p3
```



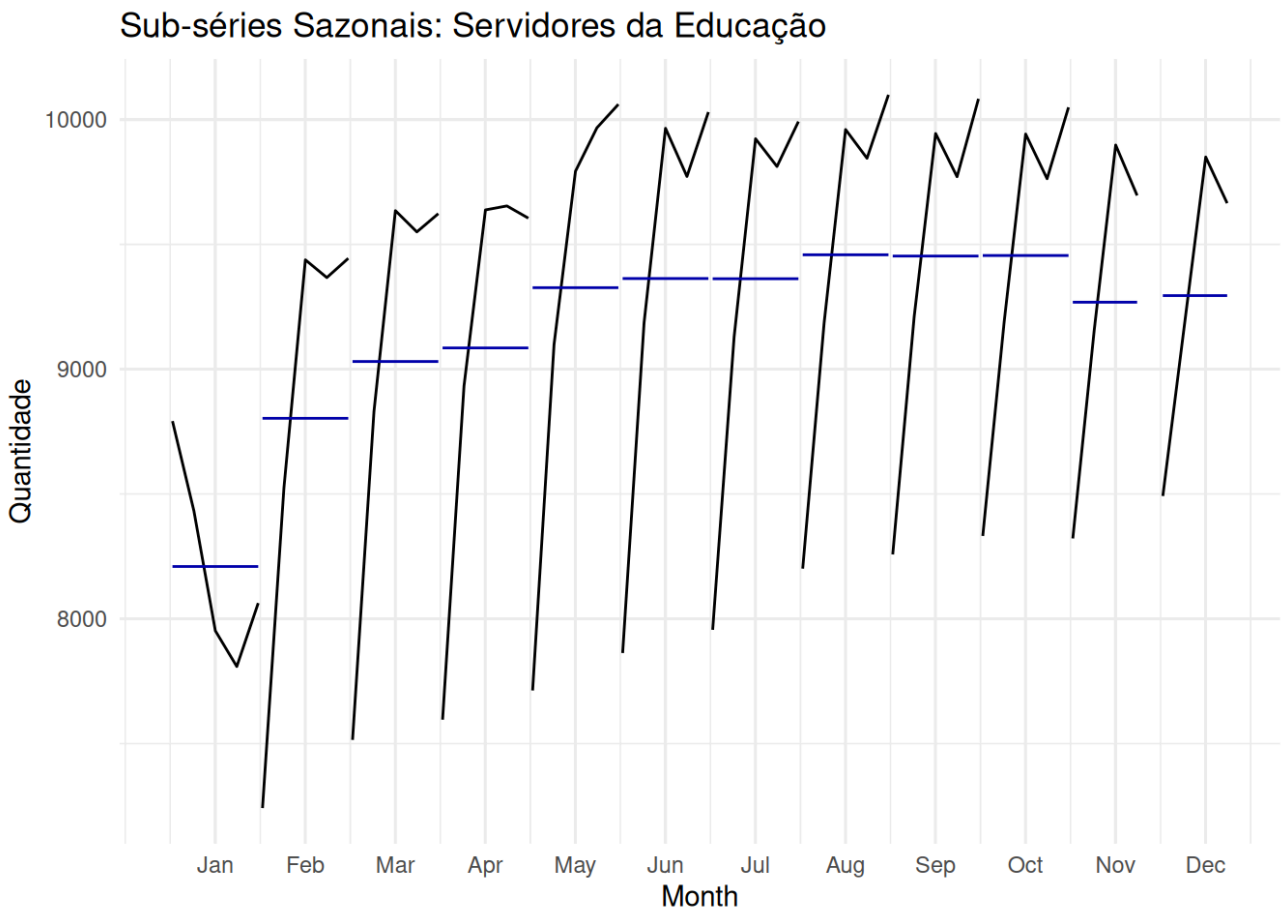

```
cat("O primeiro gráfico mostra a serie original. O segundo demonstra a tendência de cre
scimento que a serie possui. O terceiro indica a sazonalidade sendo possível observar u
m padrão de variação ciclico. Os resíduos são os eventos aleatórios da serie.")
```

```
## O primeiro gráfico mostra a serie original. O segundo demonstra a tendência de cresc
imento que a serie possui. O terceiro indica a sazonalidade sendo possível observar um
padrão de variação ciclico. Os resíduos são os eventos aleatórios da serie.
```

```
p4 <- ggsubseriesplot(serie_ts) +
  ggtitle("Sub-séries Sazonais: Servidores da Educação") +
  ylab("Quantidade") +
  theme_minimal()

cat("BoxPlot por mês\n")
```

```
## BoxPlot por mês
```

```
p4
```




```
cat("Esse gráfico agrupa os meses de todos os anos, a linha azul é a média de cada mês
indicando que as contratações de servidores da Secretaria de Educação crescem de fevere
iro a julho, estabilizam de agosto a outubro e decrescem de novembro a janeiro.")
```

```
## Esse gráfico agrupa os meses de todos os anos, a linha azul é a média de cada mês in
dicando que as contratações de servidores da Secretaria de Educação crescem de fevereir
o a julho, estabilizam de agosto a outubro e decrescem de novembro a janeiro.
```

```
#Teste de estacionariedade

teste_adf <- adf.test(serie_ts)
print(teste_adf)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  serie_ts
## Dickey-Fuller = -2.9121, Lag order = 3, p-value = 0.2065
## alternative hypothesis: stationary
```

```
if (teste_adf$p.value > 0.05) {
  cat("Conclusão: Não rejeitamos H0 - a série provavelmente NÃO é estacionária\n")
  cat("Será necessário diferenciar a série (I(d) com d >= 1)\n")
} else {
  cat("Conclusão: Rejeitamos H0 - a série provavelmente é estacionária\n")
}
```

```
## Conclusão: Não rejeitamos H0 - a série provavelmente NÃO é estacionária
## Será necessário diferenciar a série (I(d) com d >= 1)
```

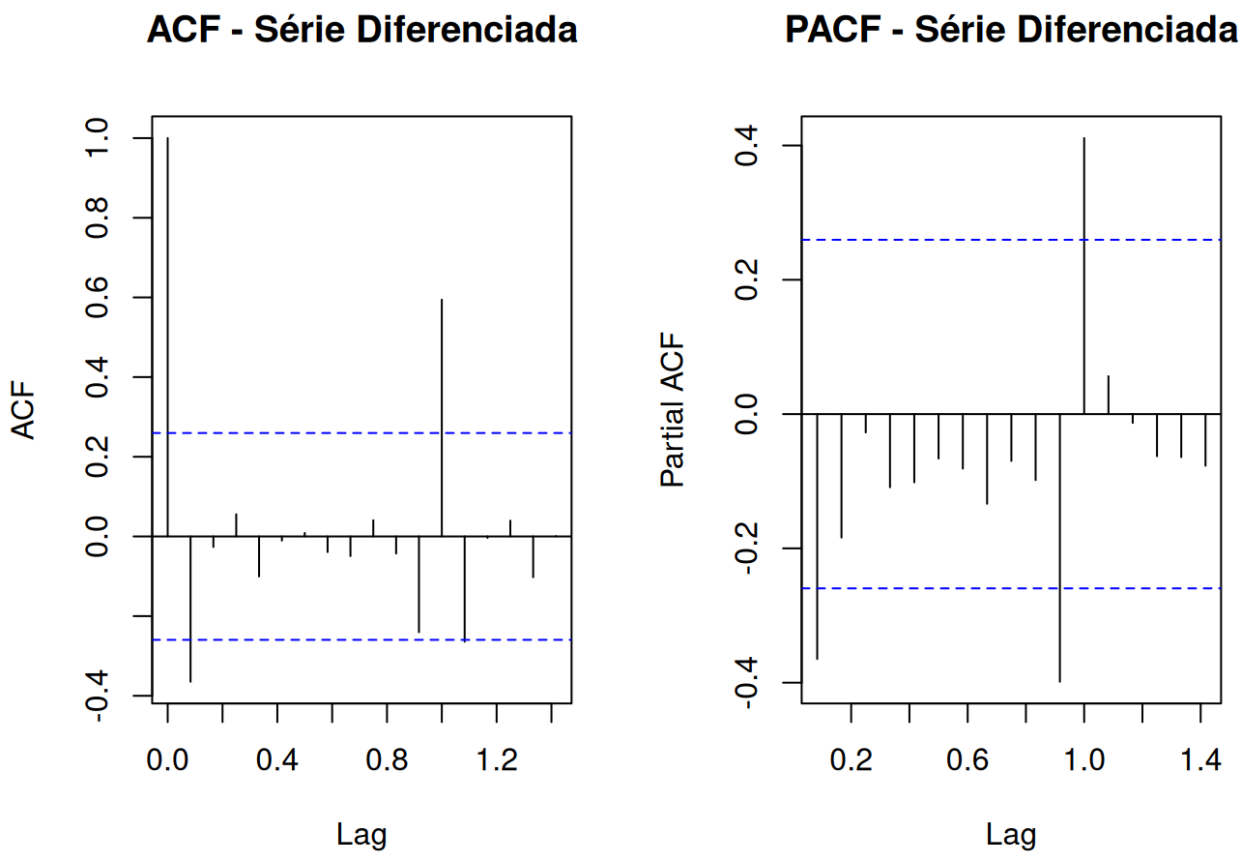
```
#Diferenciação da séria

#Aplicando a 1ª diferenciação na série
serie_diff <- diff(serie_ts)

#Teste de Estacionariedade na série diferenciada
teste_adf_diff <- adf.test(na.omit(serie_diff))
print(teste_adf_diff$p.value)
```

```
## [1] 0.01
```

```
if (teste_adf_diff$p.value > 0.05) {
  cat("Conclusão: Não rejeitamos H0 - a série NÃO é estacionária\n")
  cat("Será necessário diferenciar a série novamente (d = 2)\n")
} else {
  cat("Conclusão: Rejeitamos H0 - a série provavelmente é estacionária (d = 1)\n")
}
```

```
## Conclusão: Rejeitamos H0 - a série provavelmente é estacionária (d = 1)
```

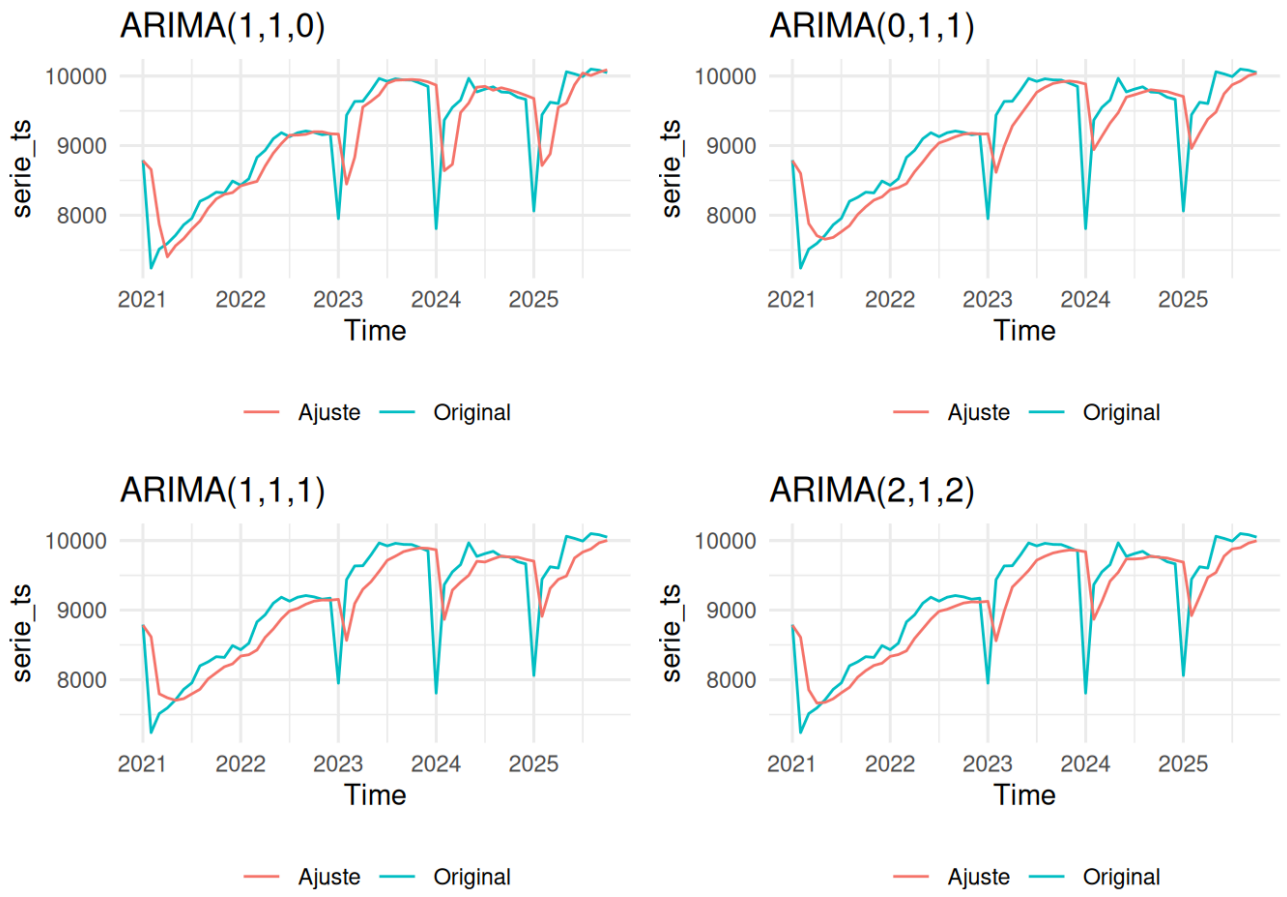
```
#ACF e PACF da série diferenciada
par(mfrow = c(1, 2))
acf(na.omit(serie_diff), main = "ACF - Série Diferenciada")
pacf(na.omit(serie_diff), main = "PACF - Série Diferenciada")
```



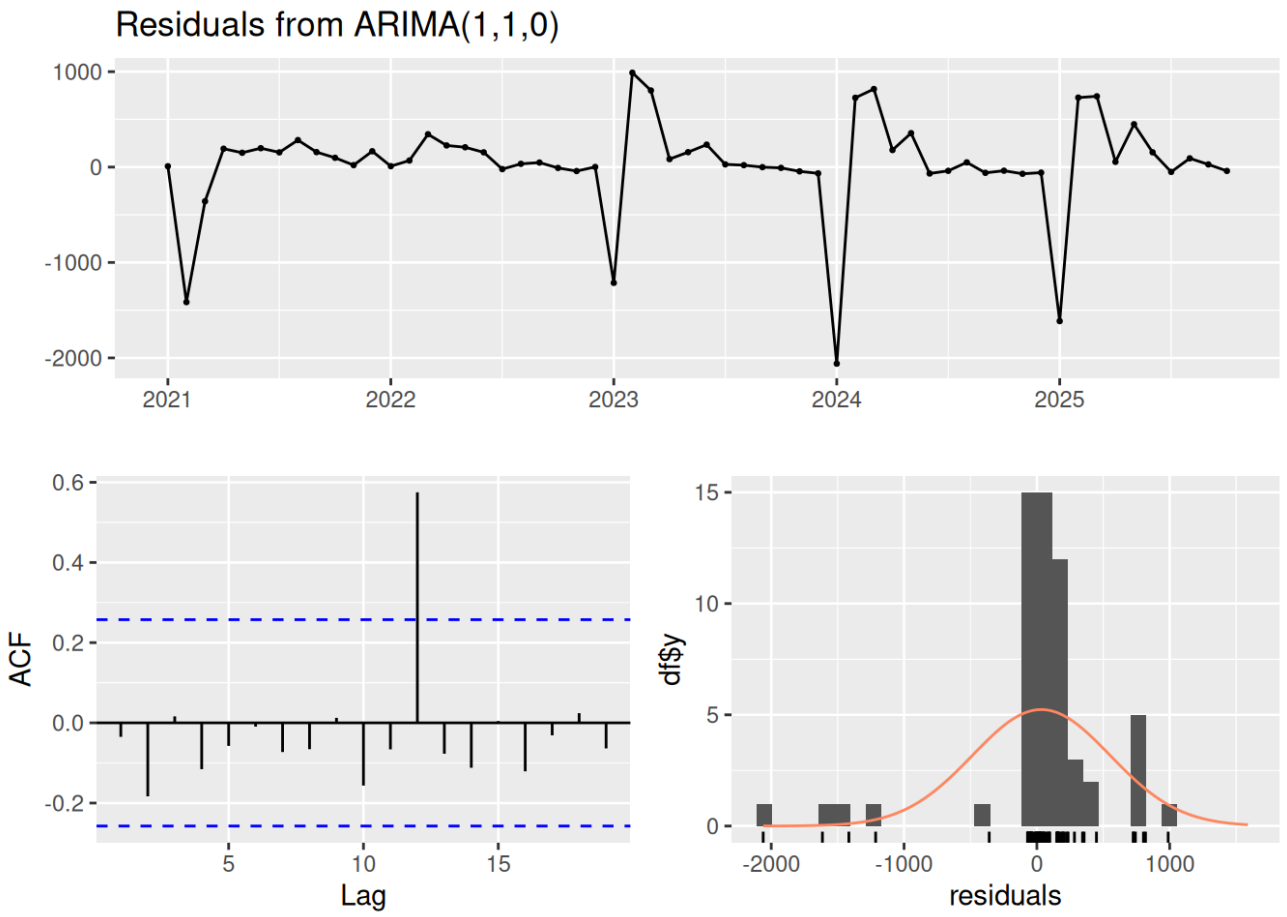

```
par(mfrow = c(1, 1))

cat("Os gráficos de ACF e PACF demonstram que existe correlação significativa, especial
mente no lag 12, indicando que o mês analisado apresenta forte dependência temporal do
mesmo mês no ano anterior.")
```

```
## Os gráficos de ACF e PACF demonstram que existe correlação significativa, especialme
nte no lag 12, indicando que o mês analisado apresenta forte dependência temporal do me
smo mês no ano anterior.
```

```r
#Ajuste de modelo - ARIMA
# Modelo 1: ARIMA(1,1,0) - Apenas Autoregressivo
modelo_arima1 <- Arima(serie_ts, order = c(1, 1, 0))

# Modelo 2: ARIMA(0,1,1) - Apenas Médias Móveis
modelo_arima2 <- Arima(serie_ts, order = c(0, 1, 1))

# Modelo 3: ARIMA(1,1,1) - AR e MA
modelo_arima3 <- Arima(serie_ts, order = c(1, 1, 1))

# Modelo 4: ARIMA(2,1,2) - parâmetro aumentado
modelo_arima4 <- Arima(serie_ts, order = c(2, 1, 2))

#Visualização

g1 <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_arima1), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("ARIMA(1,1,0)")

g2 <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_arima2), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("ARIMA(0,1,1)")

g3 <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_arima3), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("ARIMA(1,1,1)")

g4 <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_arima4), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("ARIMA(2,1,2)")

grid.arrange(g1, g2, g3, g4, ncol=2)
```

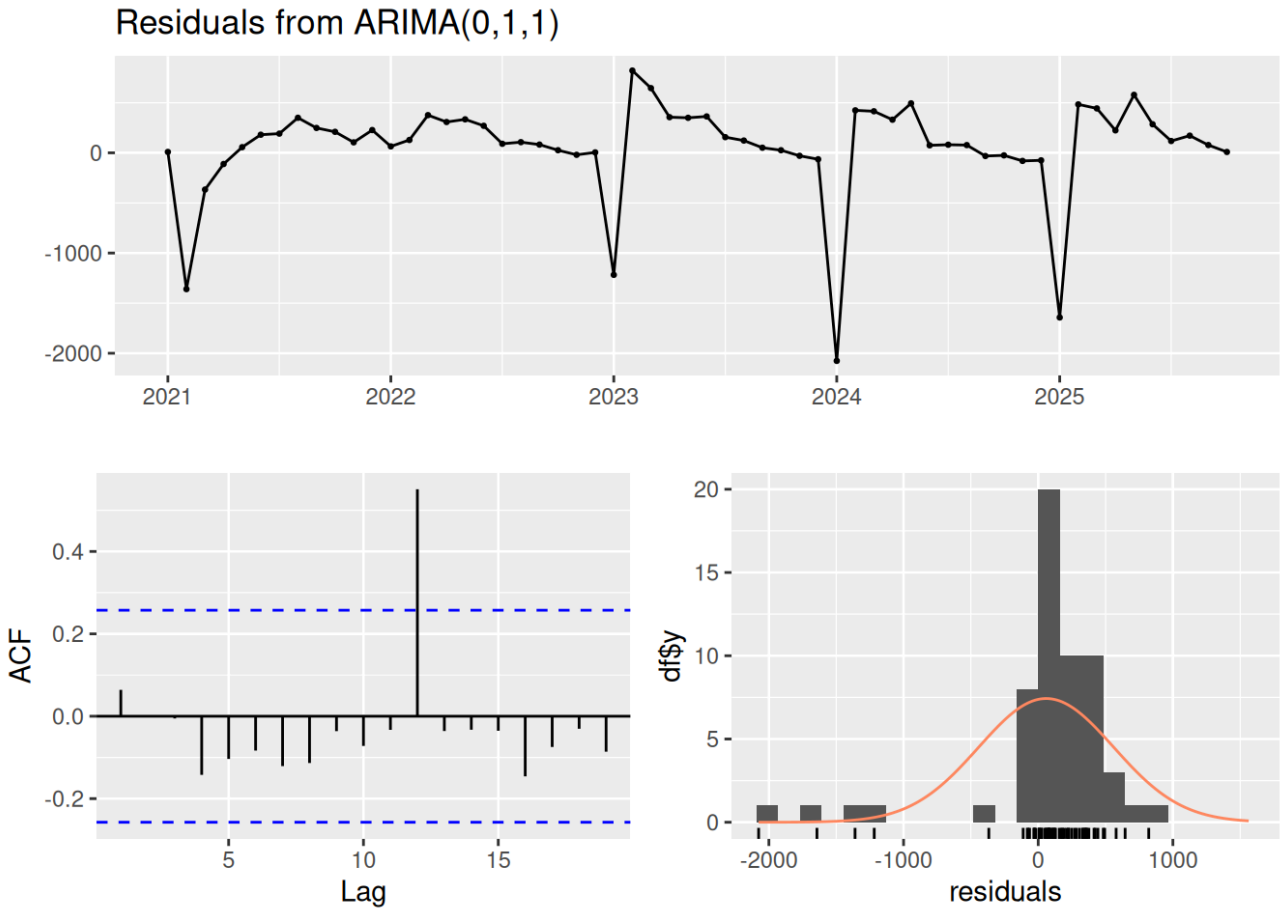



```
#Diagnostico de modelo - ARIMA

checkresiduals(modelo_arima1)
```



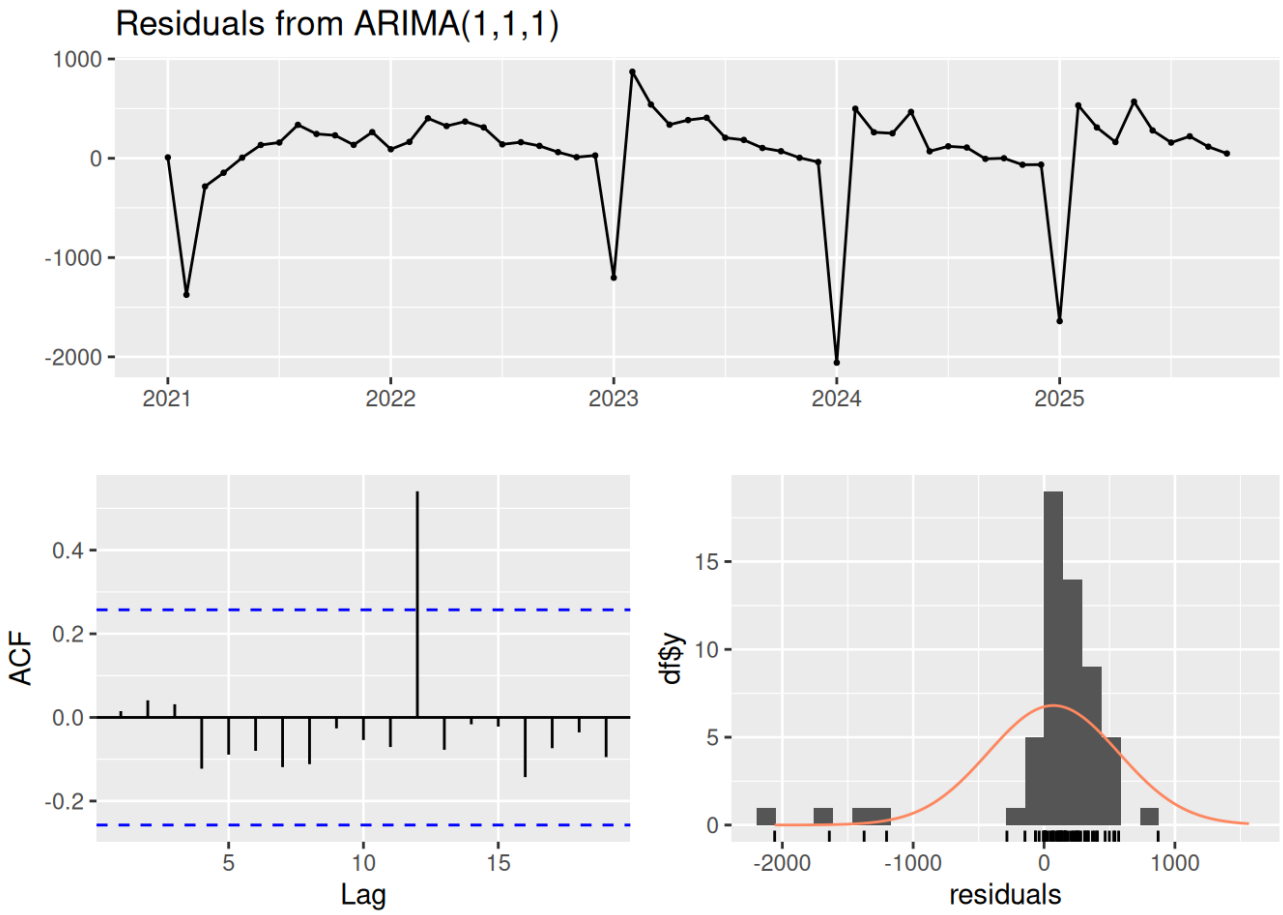
```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0)
## Q* = 31.04, df = 11, p-value = 0.001086
##
## Model df: 1.   Total lags used: 12
```

```
checkresiduals(modelo_arima2)
```



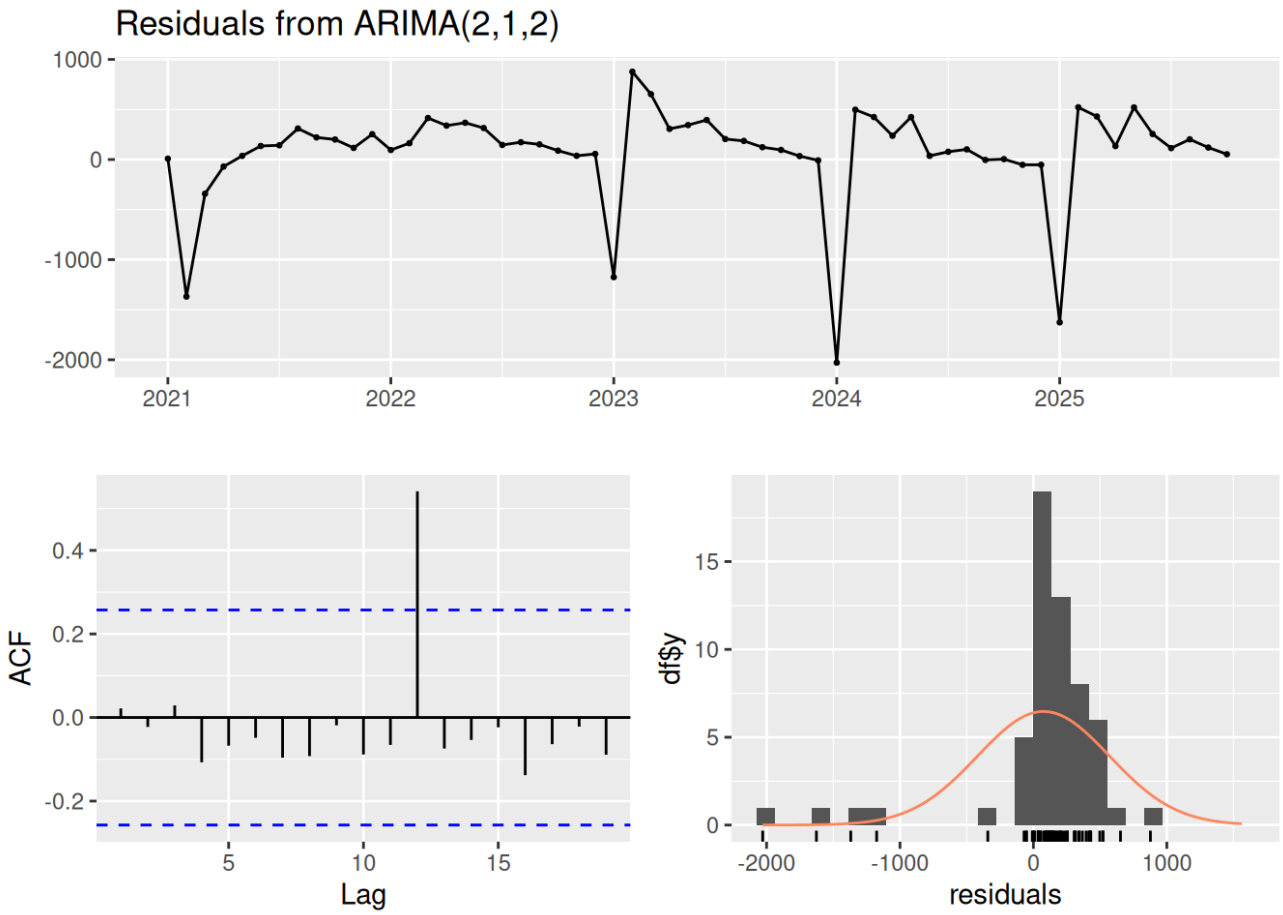

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)
## Q* = 28.145, df = 11, p-value = 0.003076
##
## Model df: 1.   Total lags used: 12
```

```
checkresiduals(modelo_arima3)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)
## Q* = 26.678, df = 10, p-value = 0.002928
##
## Model df: 2.   Total lags used: 12
```

```
checkresiduals(modelo_arima4)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,1,2)
## Q* = 25.619, df = 8, p-value = 0.00122
##
## Model df: 4.   Total lags used: 12
```

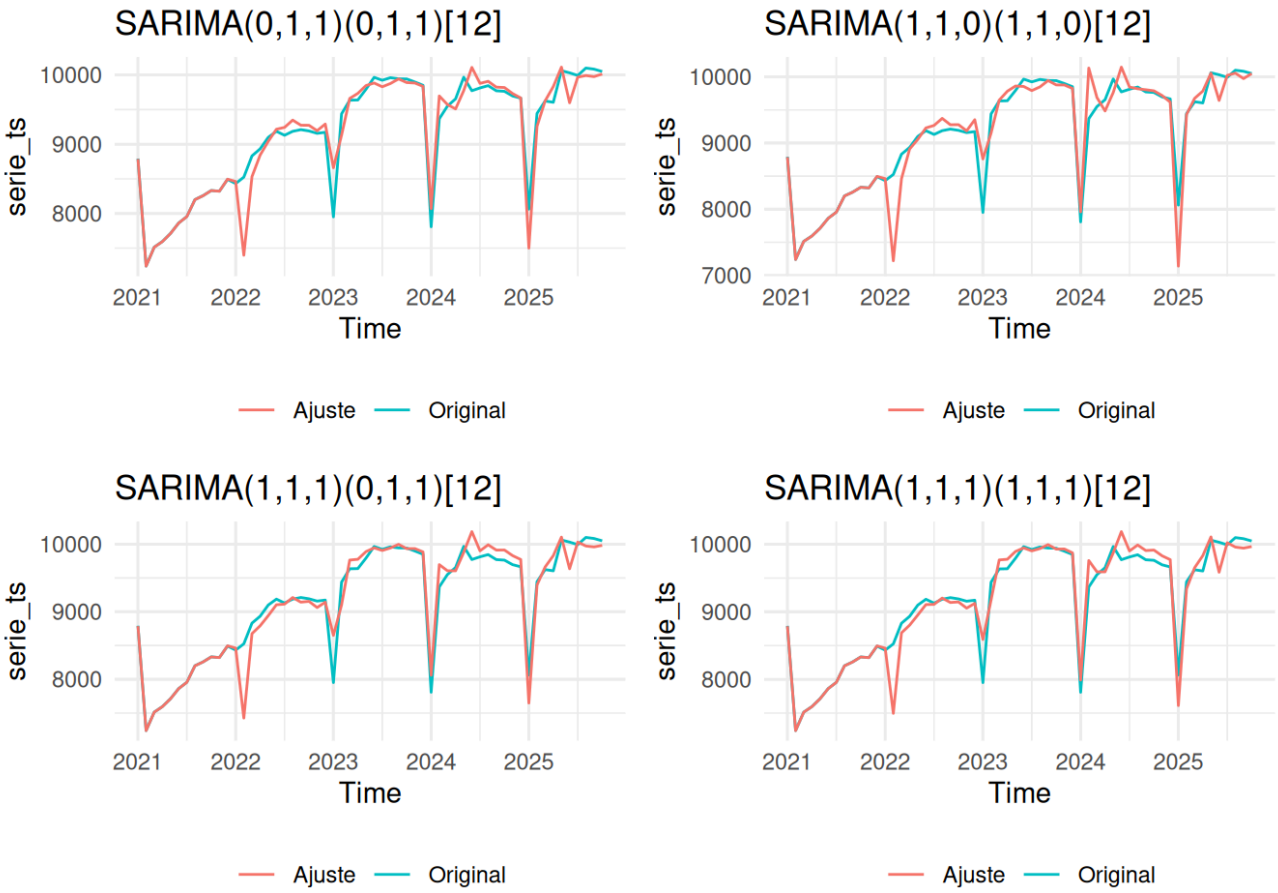
```r
lista_modelos <- list(
  "ARIMA(1,1,0)" = modelo_arima1,
  "ARIMA(0,1,1)" = modelo_arima2,
  "ARIMA(1,1,1)" = modelo_arima3,
  "ARIMA(2,1,2)" = modelo_arima4
)

for (nome in names(lista_modelos)) {
  modelo <- lista_modelos[[nome]]

  cat("\n--------------------------------------------------\n")
  cat("Avaliando:", nome, "\n")

  # 1. Teste de Ljung-Box (Independência)
  teste_ljung <- Box.test(modelo$residuals, type = "Ljung-Box")
  passou_ljung <- teste_ljung$p.value > 0.05

  cat("Ljung-Box (p-valor:", teste_ljung$p.value, ") -> ")
  if (passou_ljung) {
    cat("Resíduos Independentes\n")
  } else {
    cat("Há autocorrelação\n")
  }

  # 2. Teste de Shapiro-Wilk (Normalidade)
  teste_normalidade <- shapiro.test(modelo$residuals)
  passou_shapiro <- teste_normalidade$p.value > 0.05

  cat("Shapiro-Wilk (p-valor:", teste_normalidade$p.value, ") -> ")
  if (passou_shapiro) {
    cat("Distribuição Normal\n")
  } else {
    cat("Não é Normal\n")
  }

  # 3. Resultado final do modelo
  cat("RESULTADO: ")
  if (passou_ljung && passou_shapiro) {
    cat(" O modelo provavelmente é adequado\n")
  } else {
    cat("O modelo provavelmente é inadequado\n")
  }
}
```

```
##
## ---------------------------------------------------
## Avaliando: ARIMA(1,1,0)
## Ljung-Box (p-valor: 0.7850497 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 4.146086e-09 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ---------------------------------------------------
## Avaliando: ARIMA(0,1,1)
## Ljung-Box (p-valor: 0.6173437 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 8.426062e-10 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ---------------------------------------------------
## Avaliando: ARIMA(1,1,1)
## Ljung-Box (p-valor: 0.9067414 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 3.82661e-10 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ---------------------------------------------------
## Avaliando: ARIMA(2,1,2)
## Ljung-Box (p-valor: 0.8650204 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 3.99786e-10 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
```

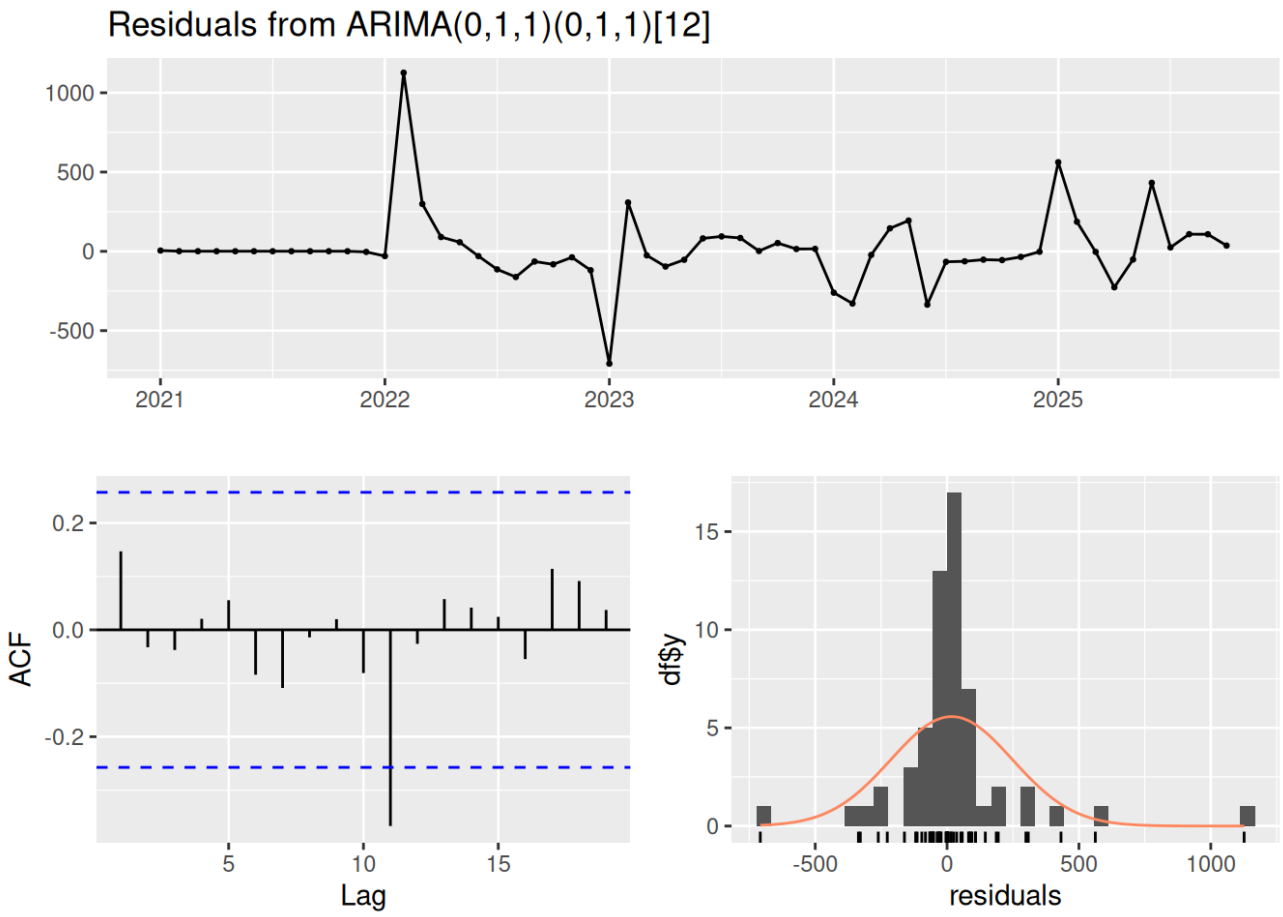
```
print("Nenhum modelo ARIMA foi considerado adequado, continuemos para SARIMA")
```

```
## [1] "Nenhum modelo ARIMA foi considerado adequado, continuemos para SARIMA"
```

```r
# Ajuste de modelo - SARIMA

# Modelo 1: SARIMA(0,1,1)(0,1,1)[12]
modelo_sarima1 <- Arima(serie_ts, order = c(0, 1, 1), seasonal = c(0, 1, 1))

# Modelo 2: SARIMA(1,1,0)(1,1,0)[12]
modelo_sarima2 <- Arima(serie_ts, order = c(1, 1, 0), seasonal = c(1, 1, 0))

# Modelo 3: SARIMA(1,1,1)(0,1,1)[12]
modelo_sarima3 <- Arima(serie_ts, order = c(1, 1, 1), seasonal = c(0, 1, 1))

# Modelo 4: SARIMA(1,1,1)(1,1,1)[12]
modelo_sarima4 <- Arima(serie_ts, order = c(1, 1, 1), seasonal = c(1, 1, 1))

#Visualização
g1S <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_sarima1), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("SARIMA(0,1,1)(0,1,1)[12]")

g2S <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_sarima2), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("SARIMA(1,1,0)(1,1,0)[12]")

g3S <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_sarima3), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("SARIMA(1,1,1)(0,1,1)[12]")

g4S <- autoplot(serie_ts, series="Original") +
  autolayer(fitted(modelo_sarima4), series="Ajuste") +
  theme_minimal() +
  theme(legend.position="bottom", legend.title=element_blank()) +
  ggtitle("SARIMA(1,1,1)(1,1,1)[12]")

grid.arrange(g1S, g2S, g3S, g4S, ncol=2)
```

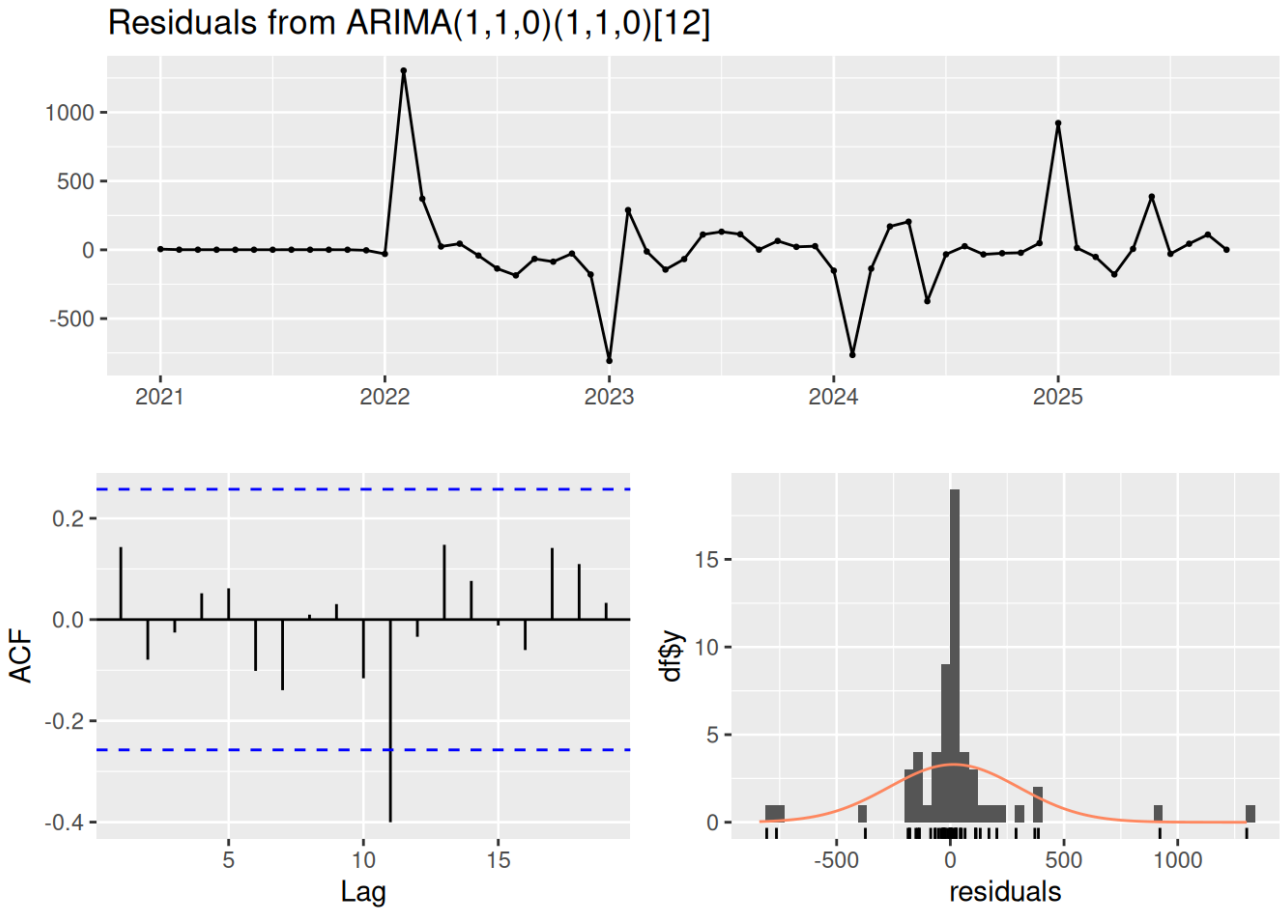



```
# Diagnóstico de modelo - SARIMA

checkresiduals(modelo_sarima1)
```



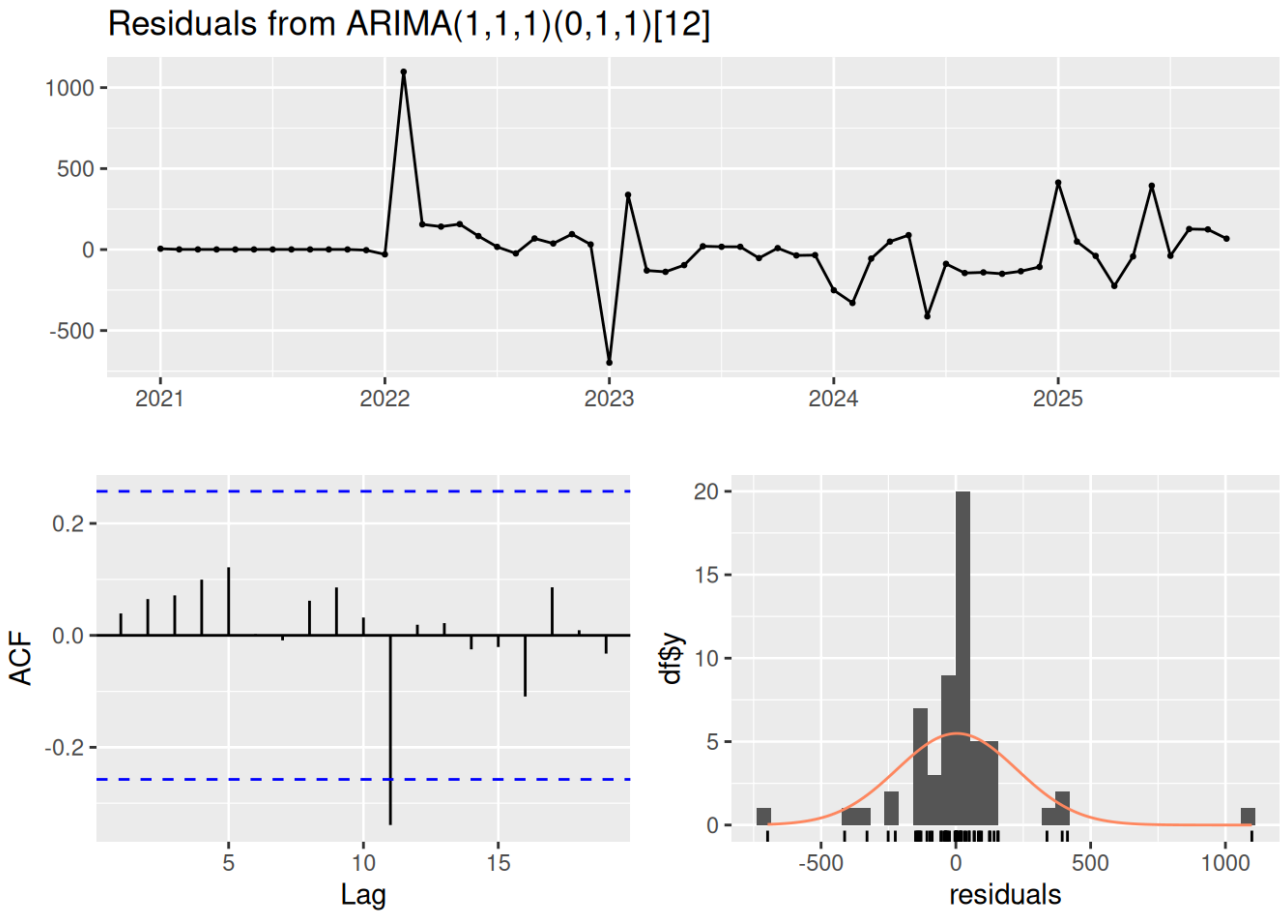
```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
## Q* = 13.528, df = 10, p-value = 0.1957
##
## Model df: 2.   Total lags used: 12
```

```
checkresiduals(modelo_sarima2)
```



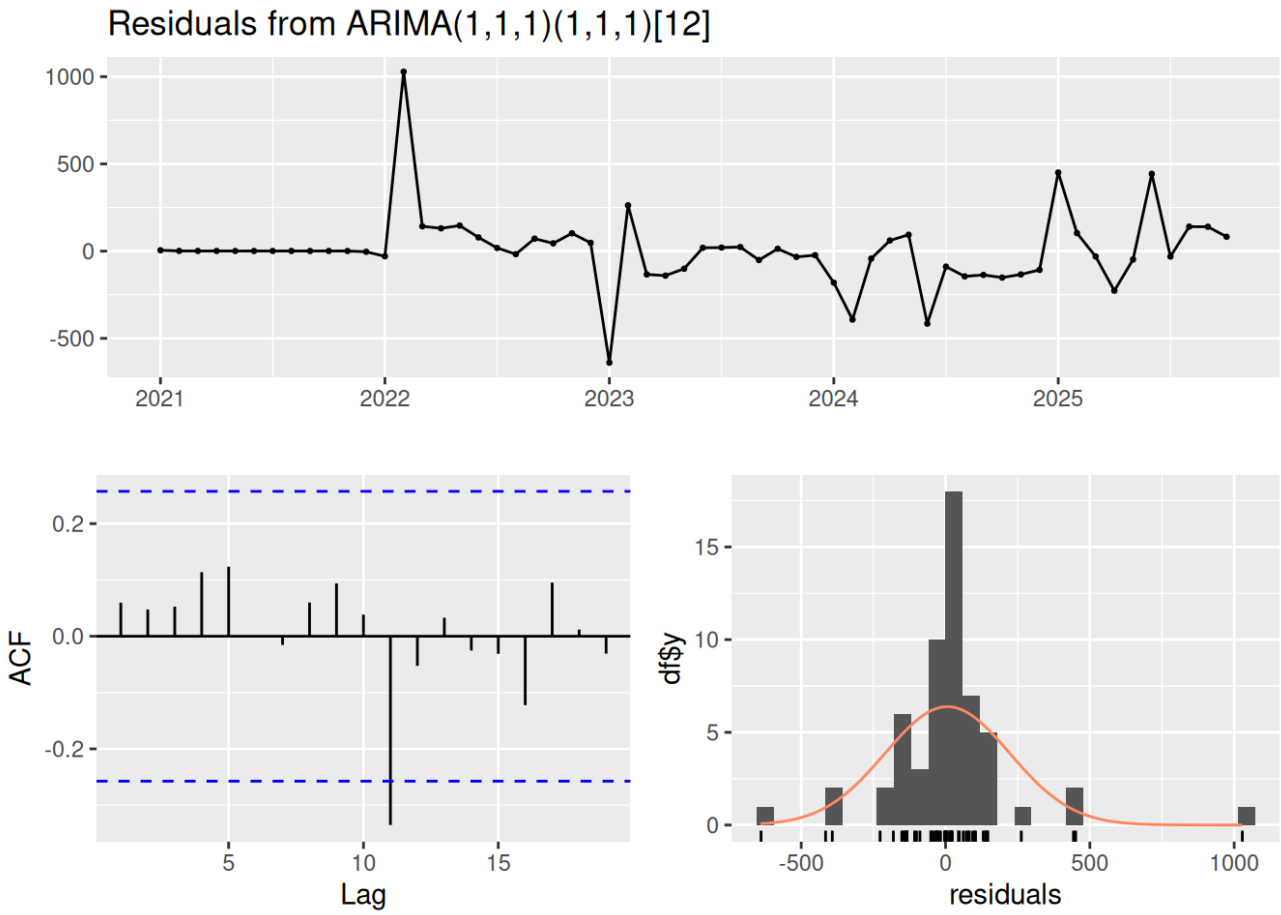

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0)(1,1,0)[12]
## Q* = 17.113, df = 10, p-value = 0.0719
##
## Model df: 2.   Total lags used: 12
```

```
checkresiduals(modelo_sarima3)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(0,1,1)[12]
## Q* = 11.68, df = 9, p-value = 0.232
##
## Model df: 3.   Total lags used: 12
```

```
checkresiduals(modelo_sarima4)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(1,1,1)[12]
## Q* = 11.891, df = 8, p-value = 0.1561
##
## Model df: 4.   Total lags used: 12
```

```r
lista_modelos_sarima <- list(
  "SARIMA(0,1,1)(0,1,1)[12]" = modelo_sarima1,
  "SARIMA(1,1,0)(1,1,0)[12]" = modelo_sarima2,
  "SARIMA(1,1,1)(0,1,1)[12]" = modelo_sarima3,
  "SARIMA(1,1,1)(1,1,1)[12]" = modelo_sarima4
)

for (nome in names(lista_modelos_sarima)) {
  modelo_sar <- lista_modelos_sarima[[nome]]

  cat("\n--------------------------------------------------\n")
  cat("Avaliando:", nome, "\n")

  # 1. Teste de Ljung-Box (Independência) - Lag ajustado para 24 (2 anos)
  teste_ljungS <- Box.test(modelo_sar$residuals, lag = 24, type = "Ljung-Box")
  passou_ljungS <- teste_ljungS$p.value > 0.05

  cat("Ljung-Box (p-valor:", teste_ljungS$p.value, ") -> ")
  if (passou_ljungS) {
    cat("Resíduos Independentes\n")
  } else {
    cat("Há autocorrelação\n")
  }

  # 2. Teste de Shapiro-Wilk (Normalidade)
  teste_normalidadeS <- shapiro.test(modelo_sar$residuals)
  passou_shapiroS <- teste_normalidadeS$p.value > 0.05

  cat("Shapiro-Wilk (p-valor:", teste_normalidadeS$p.value, ") -> ")
  if (passou_shapiroS) {
    cat("Distribuição Normal\n")
  } else {
    cat("Não é Normal\n")
  }

  # 3. Resultado final do modelo
  cat("RESULTADO: ")
  if (passou_ljungS && passou_shapiroS) {
    cat("O modelo provavelmente é adequado\n")
  } else {
    cat("O modelo provavelmente é inadequado\n")
  }
}
```

```
##
## ------------------------------------------------
## Avaliando: SARIMA(0,1,1)(0,1,1)[12]
## Ljung-Box (p-valor: 0.6872937 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 3.530034e-08 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,0)(1,1,0)[12]
## Ljung-Box (p-valor: 0.05108093 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 2.400644e-09 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,1)(0,1,1)[12]
## Ljung-Box (p-valor: 0.7054113 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 6.230272e-08 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,1)(1,1,1)[12]
## Ljung-Box (p-valor: 0.597249 ) -> Resíduos Independentes
## Shapiro-Wilk (p-valor: 1.786471e-07 ) -> Não é Normal
## RESULTADO: O modelo provavelmente é inadequado
```

```
print("Nenhum modelo SARIMA foi considerado adequado, faremos transformação box-cox par
a estabilizar a variância")
```

```
## [1] "Nenhum modelo SARIMA foi considerado adequado, faremos transformação box-cox pa
ra estabilizar a variância"
```

```r
##Box-Cox

# Ajustando os modelos SARIMA usando a transformação de Box-Cox

modelo_sarima1_bc <- Arima(serie_ts, order = c(0, 1, 1), seasonal = c(0, 1, 1), lambda
= "auto")
modelo_sarima2_bc <- Arima(serie_ts, order = c(1, 1, 0), seasonal = c(1, 1, 0), lambda
= "auto")
modelo_sarima3_bc <- Arima(serie_ts, order = c(1, 1, 1), seasonal = c(0, 1, 1), lambda
= "auto")
modelo_sarima4_bc <- Arima(serie_ts, order = c(1, 1, 1), seasonal = c(1, 1, 1), lambda
= "auto")

# Criando a lista com os novos modelos transformados
lista_sarima_bc <- list(
  "SARIMA(0,1,1)(0,1,1) + BC" = modelo_sarima1_bc,
  "SARIMA(1,1,0)(1,1,0) + BC" = modelo_sarima2_bc,
  "SARIMA(1,1,1)(0,1,1) + BC" = modelo_sarima3_bc,
  "SARIMA(1,1,1)(1,1,1) + BC" = modelo_sarima4_bc
)

for (nome in names(lista_sarima_bc)) {
  modelo_atual <- lista_sarima_bc[[nome]]

  cat("\n--------------------------------------------------\n")
  cat("Avaliando:", nome, "\n")

  # Teste de Normalidade
  teste_normalidade <- shapiro.test(modelo_atual$residuals)

  cat("p-valor Shapiro-Wilk:", teste_normalidade$p.value, "\n")

  if (teste_normalidade$p.value > 0.05) {
    cat("Resíduos normais\n")
  } else {
    cat("Resíduos não são normais (p-valor < 0.05).\n")
  }
}
```

```
##
## ------------------------------------------------
## Avaliando: SARIMA(0,1,1)(0,1,1) + BC
## p-valor Shapiro-Wilk: 7.593574e-16
## Resíduos não são normais (p-valor < 0.05).
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,0)(1,1,0) + BC
## p-valor Shapiro-Wilk: 7.370021e-16
## Resíduos não são normais (p-valor < 0.05).
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,1)(0,1,1) + BC
## p-valor Shapiro-Wilk: 7.57103e-16
## Resíduos não são normais (p-valor < 0.05).
##
## ------------------------------------------------
## Avaliando: SARIMA(1,1,1)(1,1,1) + BC
## p-valor Shapiro-Wilk: 7.610332e-16
## Resíduos não são normais (p-valor < 0.05).
```

```
print("Mesmo com a tranformação Box-Cox a normalidade dos resíduos é rejeitada")
```

```
## [1] "Mesmo com a tranformação Box-Cox a normalidade dos resíduos é rejeitada"
```

```r
# Verificando AIC dos modelos

lista_final_comparativa <- list(
  # Modelos ARIMA
  "ARIMA(1,1,0)"               = modelo_arima1,
  "ARIMA(0,1,1)"               = modelo_arima2,
  "ARIMA(1,1,1)"               = modelo_arima3,
  "ARIMA(2,1,2)"               = modelo_arima4,

  # Modelos SARIMA
  "SARIMA(0,1,1)(0,1,1)"       = modelo_sarima1,
  "SARIMA(1,1,0)(1,1,0)"       = modelo_sarima2,
  "SARIMA(1,1,1)(0,1,1)"       = modelo_sarima3,
  "SARIMA(1,1,1)(1,1,1)"       = modelo_sarima4,

  # Modelos SARIMA (Com Box-Cox)
  "SARIMA(0,1,1)(0,1,1) + BC"  = modelo_sarima1_bc,
  "SARIMA(1,1,0)(1,1,0) + BC"  = modelo_sarima2_bc,
  "SARIMA(1,1,1)(0,1,1) + BC"  = modelo_sarima3_bc,
  "SARIMA(1,1,1)(1,1,1) + BC"  = modelo_sarima4_bc
)

# Gerando a tabela de AIC
tabela_aic_completa <- data.frame(
  Modelo = names(lista_final_comparativa),
  AIC = sapply(lista_final_comparativa, AIC)
)


tabela_aic_completa <- tabela_aic_completa[order(tabela_aic_completa$AIC), ]

print(tabela_aic_completa)
```

```
##                                            Modelo       AIC
## SARIMA(0,1,1)(0,1,1) + BC SARIMA(0,1,1)(0,1,1) + BC -900.7332
## SARIMA(1,1,1)(0,1,1) + BC SARIMA(1,1,1)(0,1,1) + BC -899.2357
## SARIMA(1,1,0)(1,1,0) + BC SARIMA(1,1,0)(1,1,0) + BC -899.2216
## SARIMA(1,1,1)(1,1,1) + BC SARIMA(1,1,1)(1,1,1) + BC -894.6045
## SARIMA(1,1,1)(0,1,1)           SARIMA(1,1,1)(0,1,1)  652.5034
## SARIMA(0,1,1)(0,1,1)           SARIMA(0,1,1)(0,1,1)  653.0052
## SARIMA(1,1,1)(1,1,1)           SARIMA(1,1,1)(1,1,1)  654.0108
## SARIMA(1,1,0)(1,1,0)           SARIMA(1,1,0)(1,1,0)  657.9785
## ARIMA(0,1,1)                           ARIMA(0,1,1)  875.6464
## ARIMA(1,1,1)                           ARIMA(1,1,1)  877.3948
## ARIMA(1,1,0)                           ARIMA(1,1,0)  878.7738
## ARIMA(2,1,2)                           ARIMA(2,1,2)  880.9059
```

```r
print("Modelo com o melhor AIC foi modelo_sarima1_bc\n")
```

```
## [1] "Modelo com o melhor AIC foi modelo_sarima1_bc\n"
```

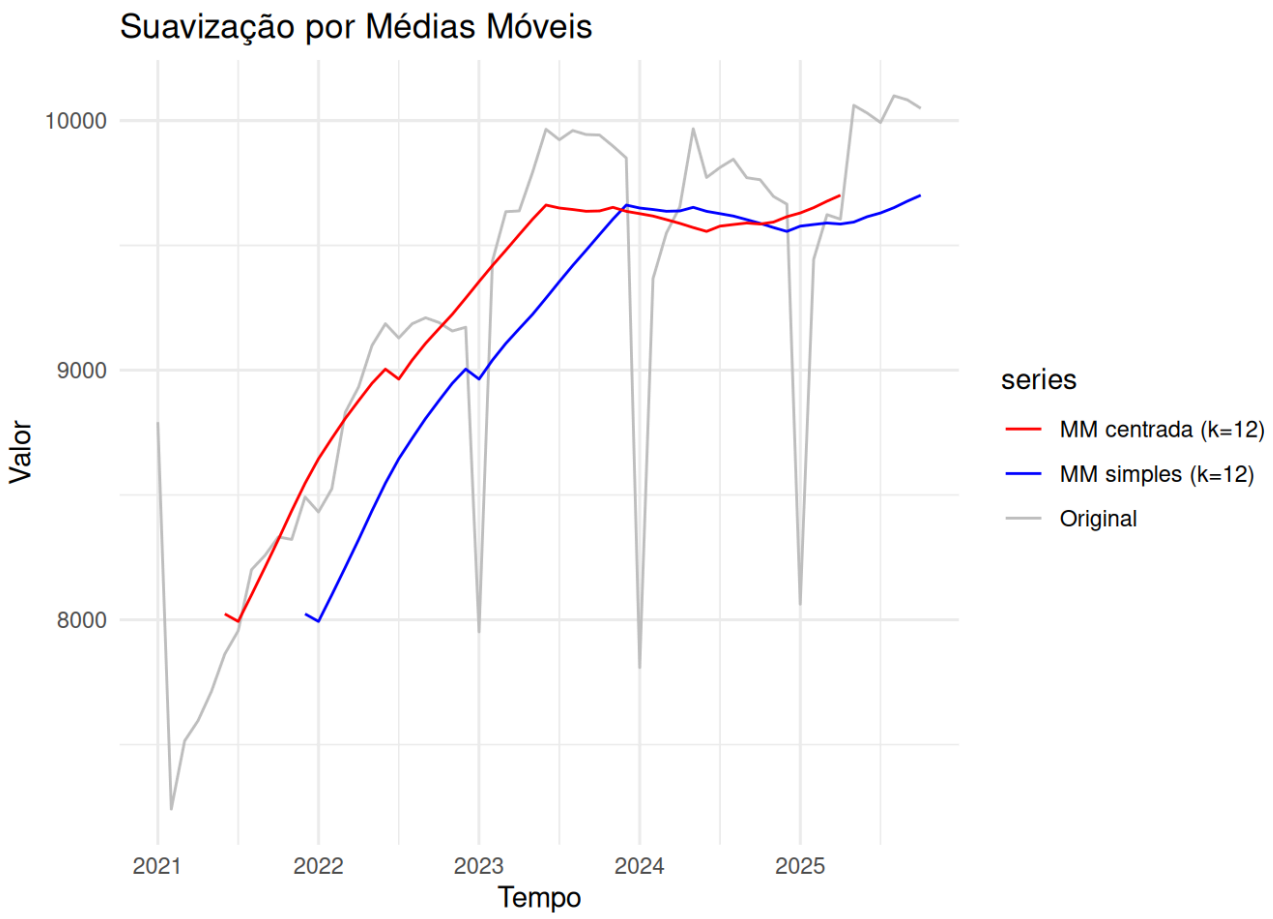
```r
cat(" Acurácia\n ")
```

```
##  Acurácia
##
```

```
accuracy(modelo_sarima1_bc)
```

```
##                     ME     RMSE      MAE      MPE     MAPE     MASE      ACF1
## Training set 471.6817 1234.573 626.2436 5.876045 7.591923 1.160645 0.6952943
```

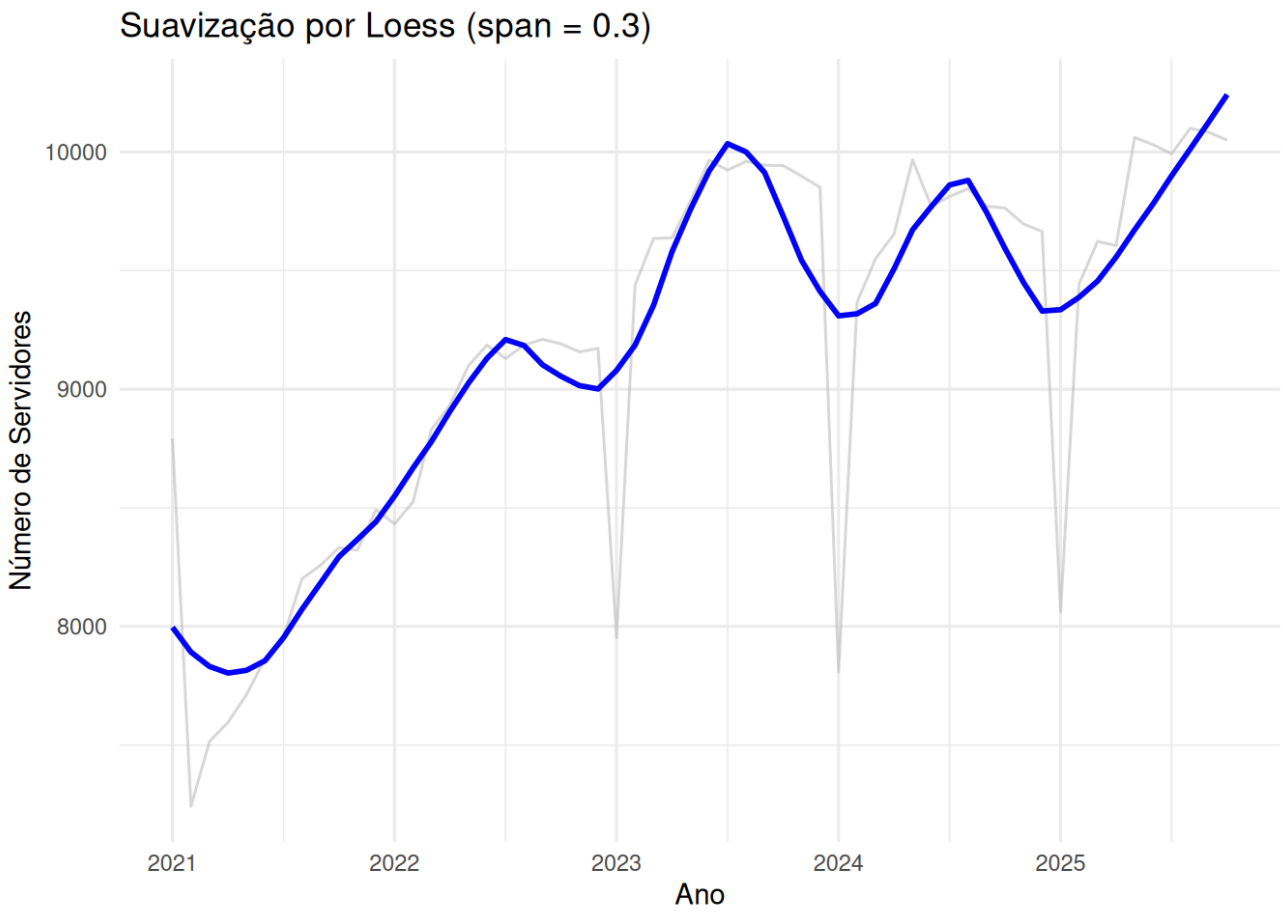
```
#MODELOS NÃO PARAMETRICOS
#Suavuavização
# Média móvel simples (janela = 12 meses) - alinhada à direita
mm_simples <- rollmean(serie_ts, k = 12, align = "right", fill = NA)

# Média móvel centrada (janela = 12) - requer duas médias de 12 para centrar
mm_centrada <- rollmean(serie_ts, k = 12, align = "center", fill = NA)

# Gráfico comparativo
autoplot(serie_ts, series = "Original") +
  autolayer(mm_simples, series = "MM simples (k=12)") +
  autolayer(mm_centrada, series = "MM centrada (k=12)") +
  ggtitle("Suavização por Médias Móveis") +
  xlab("Tempo") + ylab("Valor") +
  scale_color_manual(values = c("Original" = "gray",
                                "MM simples (k=12)" = "blue",
                                "MM centrada (k=12)" = "red")) +
  theme_minimal()
```



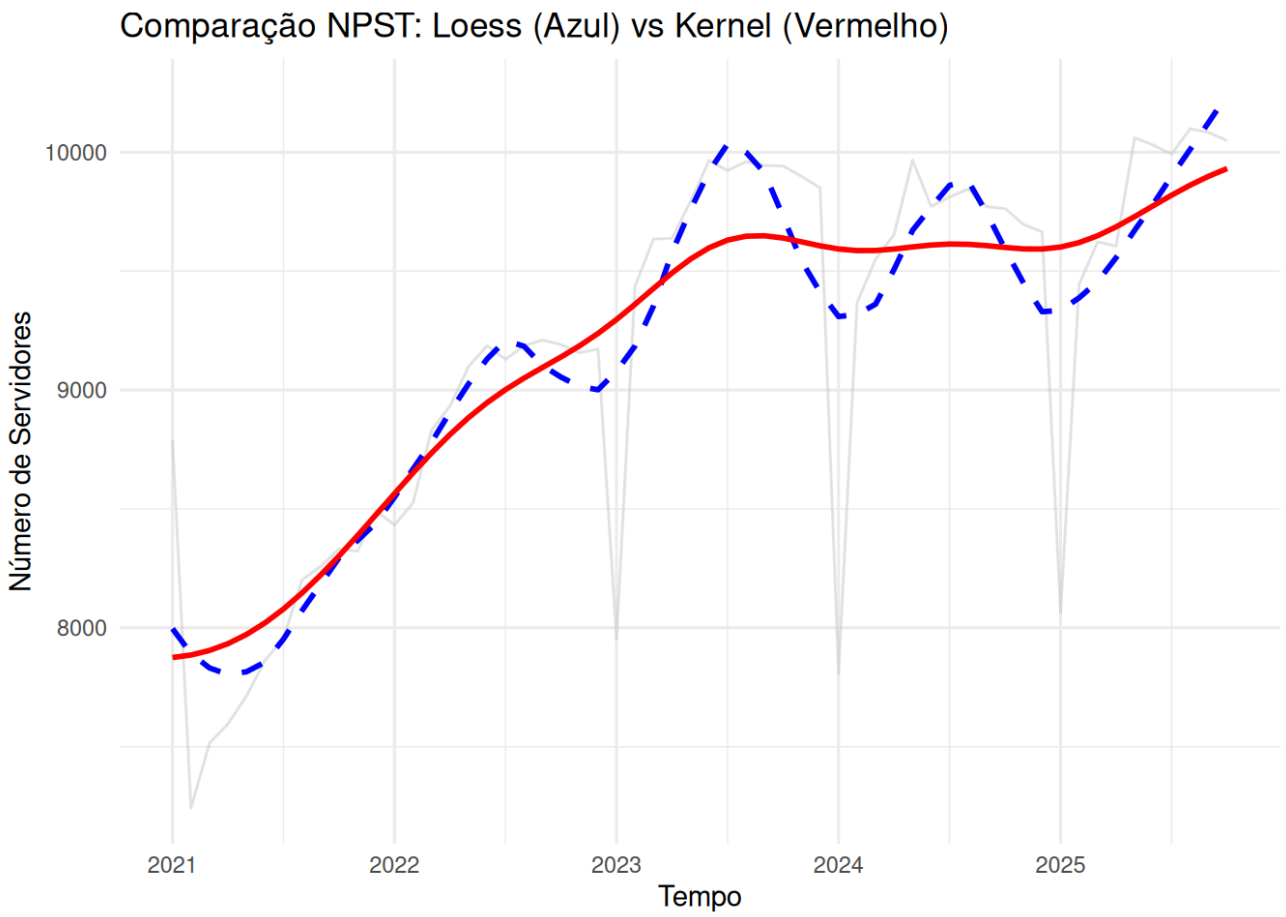
```
cat("A suavização foi usada para filtrar as oscilações sazonais e isolar a tendência. É
possível observar que após um período de alto crescimento entre 2021 e 2023, o crescime
nto no número de servidores apresentou uma estabilização a partir de 2024.")
```

```
## A suavização foi usada para filtrar as oscilações sazonais e isolar a tendência. É p
ossível observar que após um período de alto crescimento entre 2021 e 2023, o crescimen
to no número de servidores apresentou uma estabilização a partir de 2024.
```

```
#Suavização por Loess (Regressão Local)

# Ajuste loess para tendência (usando apenas o tempo)
tempo <- 1:length(serie_ts)
loess_fit <- loess(serie_ts ~ tempo, span = 0.3)
tendencia_loess <- predict(loess_fit)

df <- data.frame(
  tempo = as.numeric(time(serie_ts)),
  serie = as.numeric(serie_ts),
  tendencia_loess = tendencia_loess
)

ggplot(df, aes(x = tempo)) +
  geom_line(aes(y = serie), color = "gray", alpha = 0.6) +
  geom_line(aes(y = tendencia_loess), color = "blue", linewidth = 1) +
  scale_x_continuous(breaks = seq(2021, 2026, 1)) +
  ggtitle("Suavização por Loess (span = 0.3)") +
  xlab("Ano") + ylab("Número de Servidores") +
  theme_minimal()
```



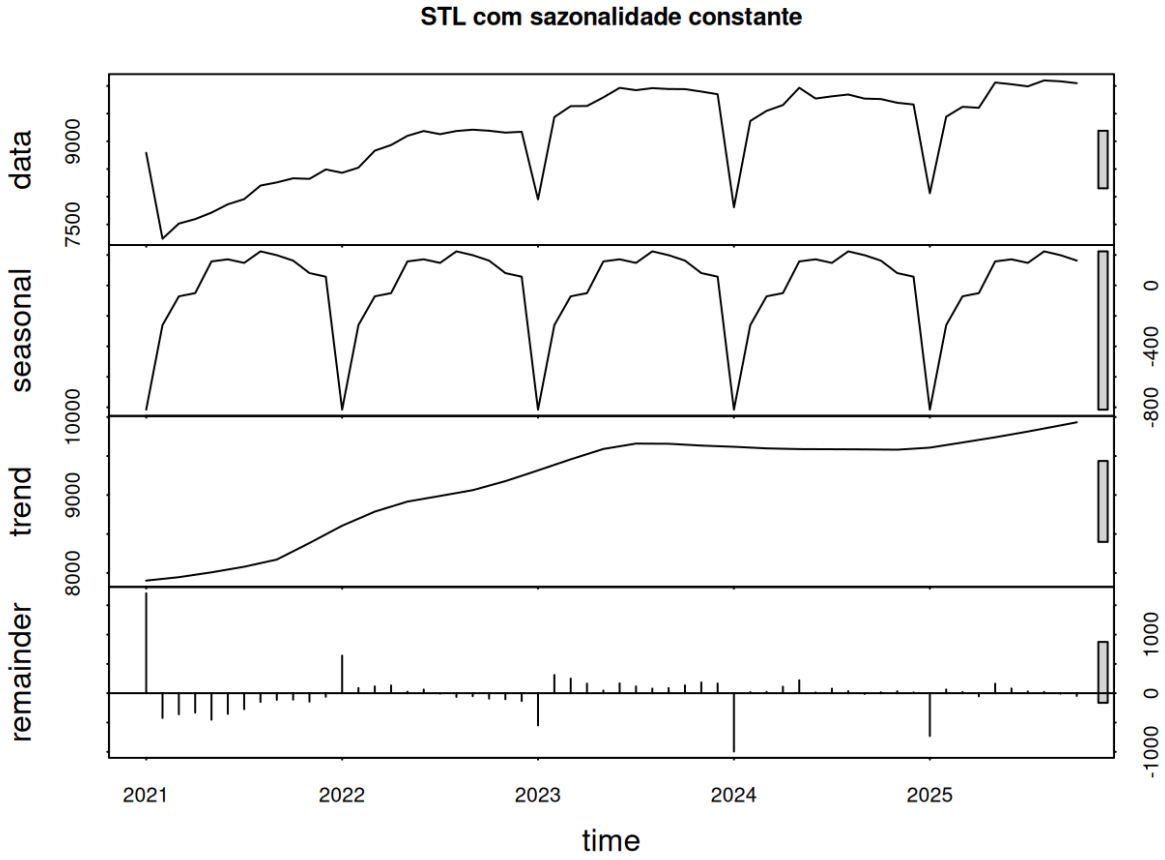
```
#Suavização por Kernel

kernel_fit <- ksmooth(tempo, as.numeric(serie_ts),
                      kernel = "normal",
                      bandwidth = 12,
                      n.points = length(serie_ts))

df$tendencia_kernel <- kernel_fit$y

#Comparando Loess e Kernel
ggplot(df, aes(x = tempo)) +
  geom_line(aes(y = serie), color = "gray", alpha = 0.4) +
  geom_line(aes(y = tendencia_loess), color = "blue", linewidth = 1, linetype = "dashe
d") +
  geom_line(aes(y = tendencia_kernel), color = "red", linewidth = 1) +
  ggtitle("Comparação NPST: Loess (Azul) vs Kernel (Vermelho)") +
  xlab("Tempo") + ylab("Número de Servidores") +
  theme_minimal()
```



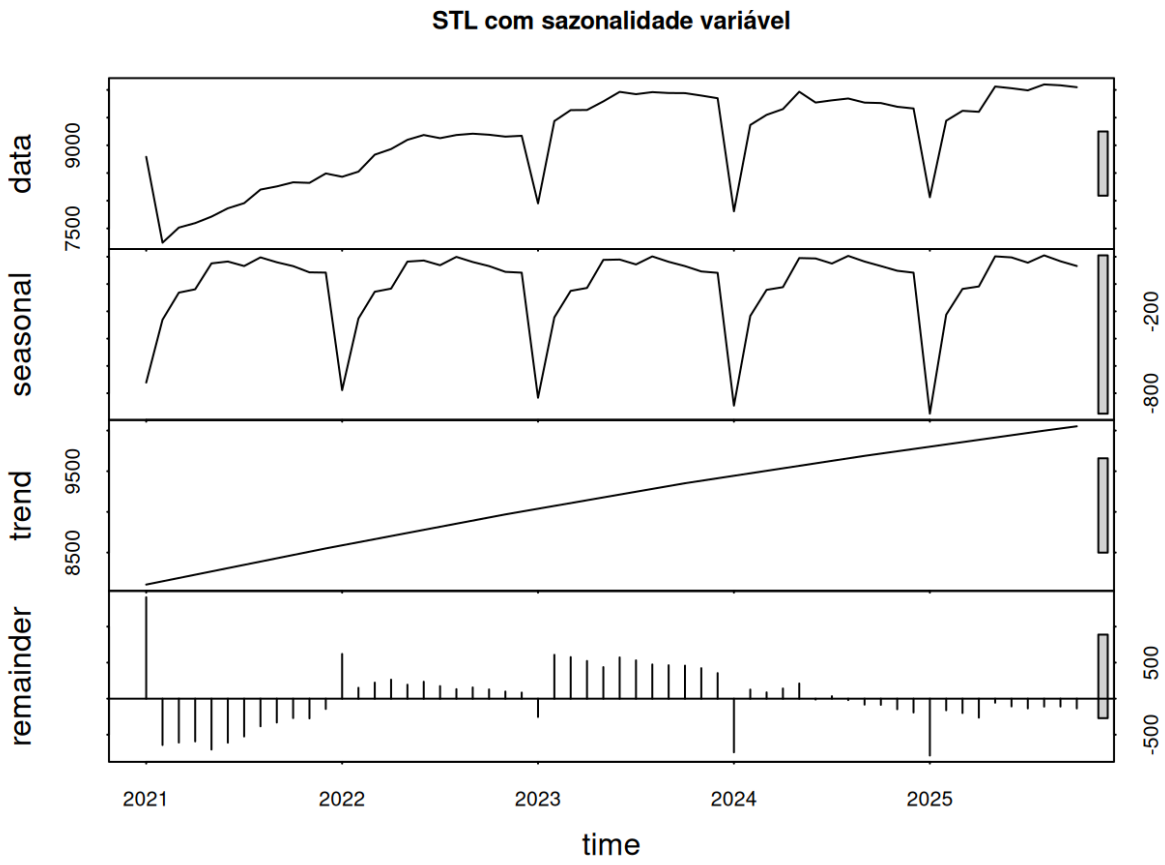

```
cat("A suavização por Loess adapta-se melhor às variações da série evidenciando as qued
as sazonais de janeiro, enquanto a suavização por Kernel demonstra uma trajetória mais
estável e suavizada do crescimento.")
```

```
## A suavização por Loess adapta-se melhor às variações da série evidenciando as quedas
sazonais de janeiro, enquanto a suavização por Kernel demonstra uma trajetória mais est
ável e suavizada do crescimento.
```

```r
#Decomposição STL (Seasonal-Trend using Loess)

dec_stl <- stl(serie_ts, s.window = "periodic")
dec_stl_var <- stl(serie_ts, s.window = 13, t.window = 101)

plot(dec_stl, main = "STL com sazonalidade constante")
```




```r
plot(dec_stl_var, main = "STL com sazonalidade variável")
```




```
cat("A decomposição STL demonstra que a sazonalidade constante é possivelmente a mais a
dequada para descrever a série temporal, indicando que provavelmente o ciclo de contrat
ações da Secretaria de Educação possui um padrão de estabilidade anual.")
```

```
## A decomposição STL demonstra que a sazonalidade constante é possivelmente a mais ade
quada para descrever a série temporal, indicando que provavelmente o ciclo de contrataç
ões da Secretaria de Educação possui um padrão de estabilidade anual.
```

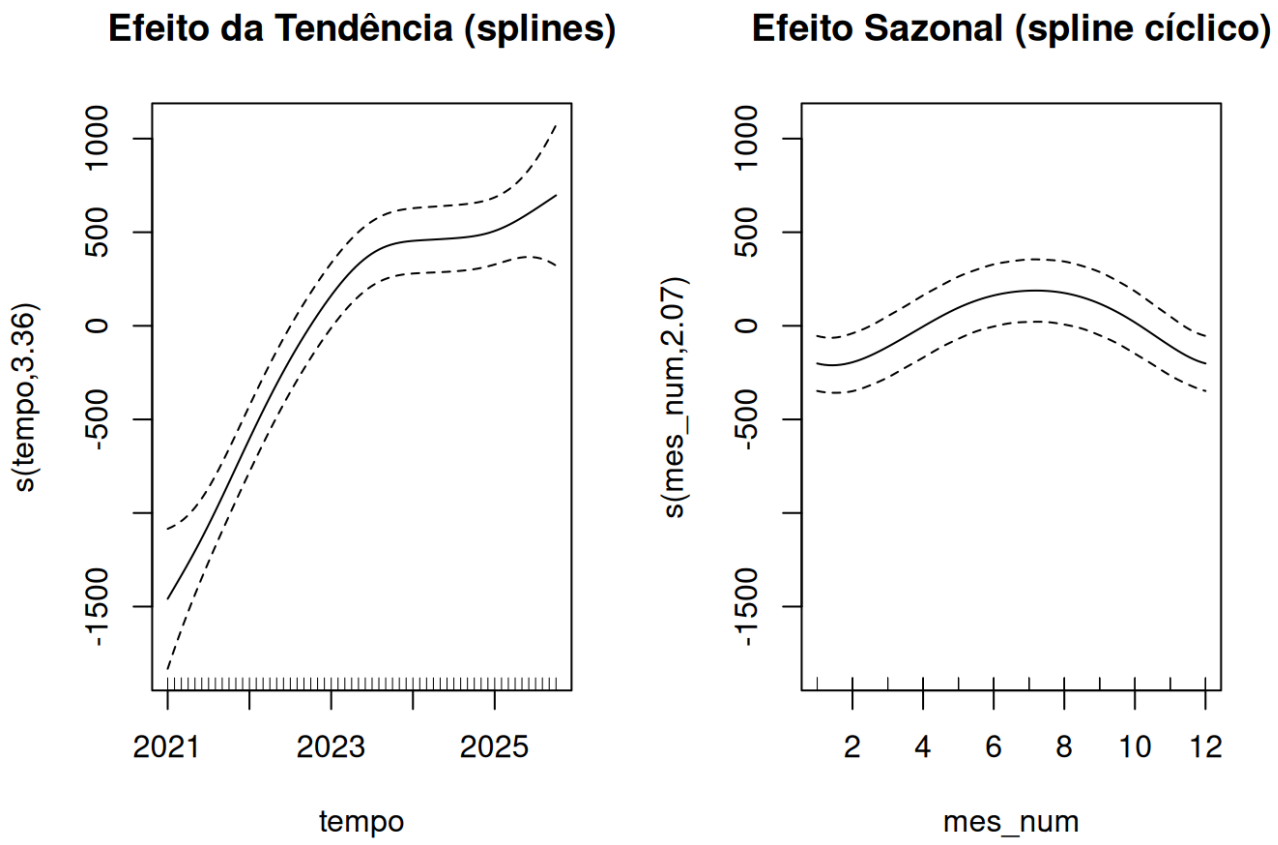
```
#Modelos Aditivos Generalizados (GAM) para Séries Temporais

dados_gam <- data.frame(
  valor   = as.numeric(serie_ts),
  tempo   = as.numeric(time(serie_ts)),
  mes_num = as.numeric(cycle(serie_ts))
)

gam_fit <- gam(valor ~ s(tempo, bs = "tp", k = 15) +
                       s(mes_num, bs = "cc", k = 12),
               data = dados_gam, method = "REML")


summary(gam_fit)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## valor ~ s(tempo, bs = "tp", k = 15) + s(mes_num, bs = "cc", k = 12)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   9172.3       56.6   162.1   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df      F p-value
## s(tempo)   3.361  4.174 30.791 < 2e-16 ***
## s(mes_num) 2.073 10.000  0.846 0.00974 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.716   Deviance explained = 74.3%
## -REML =  427.5  Scale est. = 1.858e+05  n = 58
```

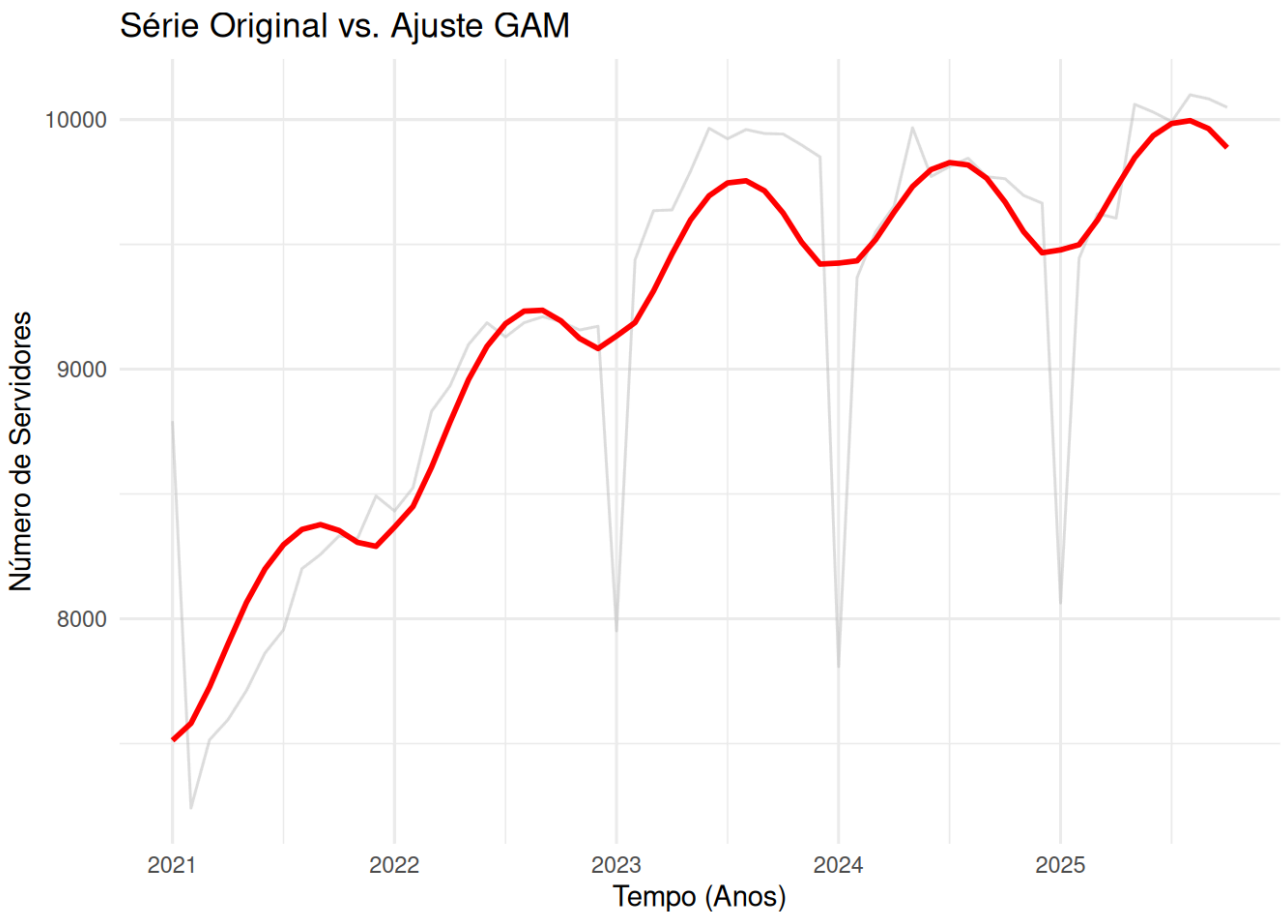
```
cat("O modelo GAM indica que os componentes de tendência e sazonalidade são estatistica
mente significativos para explicar a variação no número de servidores (p-valor < 0,05).
o modelo apresentou um desempenho de 71%.")
```

```
## O modelo GAM indica que os componentes de tendência e sazonalidade são estatisticame
nte significativos para explicar a variação no número de servidores (p-valor < 0,05). o
modelo apresentou um desempenho de 71%.
```

```
#Visualizar efeitos

par(mfrow = c(1, 2))
plot(gam_fit, select = 1, shade = TRUE, main = "Efeito da Tendência (splines)")
plot(gam_fit, select = 2, shade = TRUE, main = "Efeito Sazonal (spline cíclico)")
```

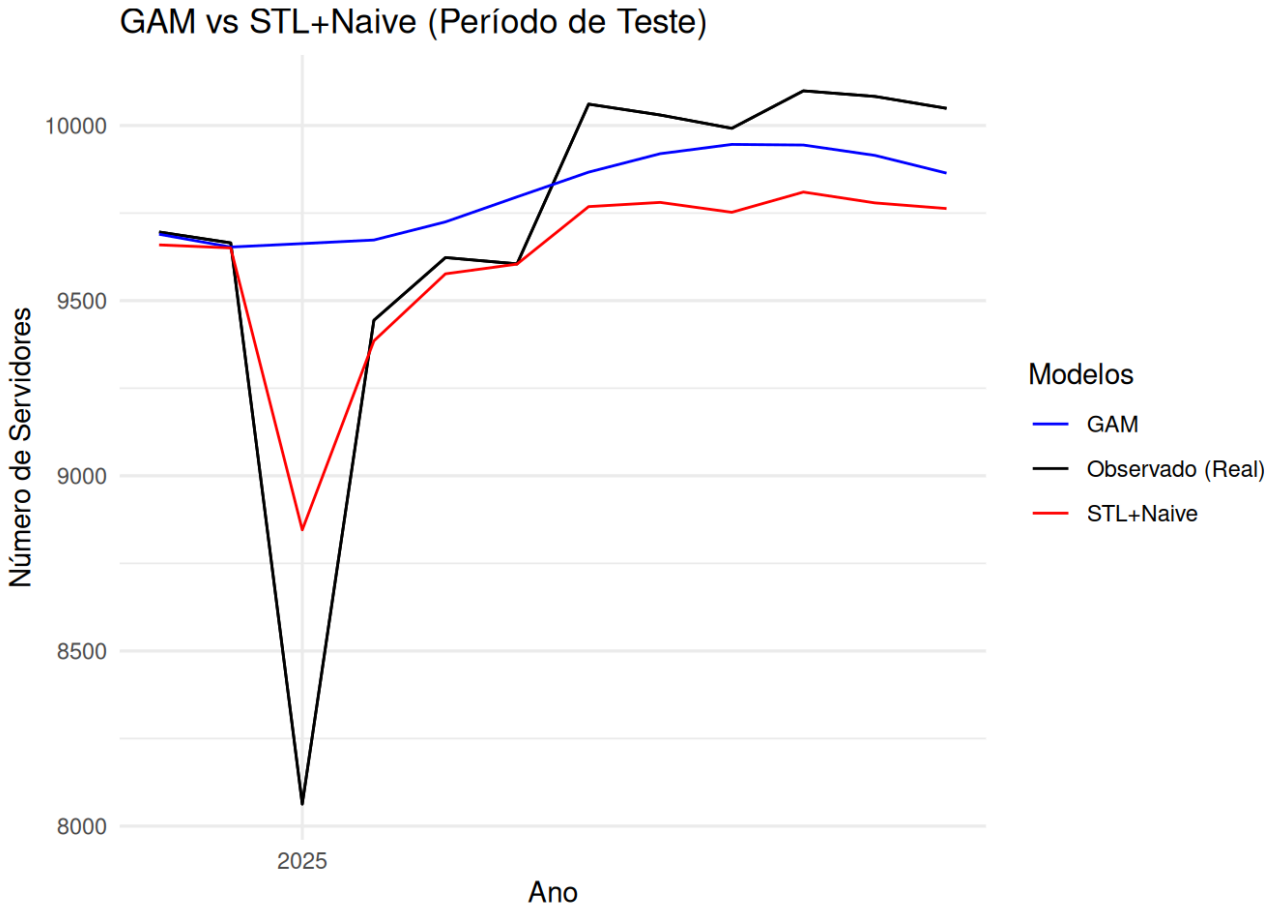



```
# Valores ajustados

dados_gam$ajustado <- fitted(gam_fit)

#Comparação com os dados originais
ggplot(dados_gam, aes(x = tempo)) +
  geom_line(aes(y = valor), color = "gray", alpha = 0.5) +
  geom_line(aes(y = ajustado), color = "red", linewidth = 1) +
  ggtitle("Série Original vs. Ajuste GAM") +
  xlab("Tempo (Anos)") +
  ylab("Número de Servidores") +
  theme_minimal()
```


Série Original vs. Ajuste GAM
10000
9000
8000
Número de Servidores
2021
2022
2023
2024
2025
Tempo (Anos)

```
#Validação com dados de treino e teste e seleção de modelo

fim_treino <- end(serie_ts) - c(1, 0)
inicio_teste <- end(serie_ts) - c(0, 11)
treino <- window(serie_ts, end = fim_treino)
teste <- window(serie_ts, start = inicio_teste)
h_teste <- length(teste)

#Reajustar GAM apenas com o Treino
dados_treino <- data.frame(
  valor = as.numeric(treino),
  tempo = as.numeric(time(treino)),
  mes_num = as.numeric(cycle(treino))
)

gam_treino <- gam(valor ~ s(tempo, bs = "tp", k = 15) +
                    s(mes_num, bs = "cc", k = 12),
                  data = dados_treino, method = "REML")


tempo_futuro_treino <- as.numeric(time(teste))

ultimo_mes_treino <- as.numeric(tail(cycle(treino), 1))
mes_futuro_treino <- rep(1:12, length.out = h_teste + ultimo_mes_treino)[(ultimo_mes_tr
eino + 1):(ultimo_mes_treino + h_teste)]

novos_treino <- data.frame(
  tempo = tempo_futuro_treino,
  mes_num = mes_futuro_treino
)

#Gerar previsões para comparação

pred_gam_treino <- predict(gam_treino, newdata = novos_treino)
pred_gam_ts <- ts(pred_gam_treino, start = start(teste), frequency = 12)

previsao_stl_treino <- stlf(treino, method = "naive", h = h_teste)

# Cálculo de erros e resultados

erro_gam <- teste - pred_gam_ts
erro_stl <- teste - previsao_stl_treino$mean

resultados <- data.frame(
  Metodo = c("GAM", "STL+Naive"),
  MAE = c(mean(abs(erro_gam)), mean(abs(erro_stl))),
  RMSE = c(sqrt(mean(erro_gam^2)), sqrt(mean(erro_stl^2))),
  MAPE = c(mean(abs(erro_gam/teste))*100, mean(abs(erro_stl/teste))*100)
)

#gráfico comparativo
autoplot(teste) +
  autolayer(teste, series = "Observado (Real)") +
  autolayer(pred_gam_ts, series = "GAM") +
  autolayer(previsao_stl_treino$mean, series = "STL+Naive") +
```

```
  scale_color_manual(values = c("Observado (Real)" = "black",
                                "GAM" = "blue",
                                "STL+Naive" = "red")) +

  ggtitle("GAM vs STL+Naive (Período de Teste)") +
  xlab("Ano") + ylab("Número de Servidores") +
  guides(color = guide_legend(title = "Modelos")) +
  theme_minimal()
```




```
print(resultados)
```

```
##      Metodo      MAE     RMSE     MAPE
## 1       GAM 249.9181 482.8592 2.836720
## 2 STL+Naive 216.6932 300.3803 2.321879
```

```
cat("O melhor modelo foi o modelo STL+Naive com um MAPE de 2.321. De todos os modelos t
estados (paramétricos e não paramétricos) este foi o melhor resultado.")
```

```
## O melhor modelo foi o modelo STL+Naive com um MAPE de 2.321. De todos os modelos tes
tados (paramétricos e não paramétricos) este foi o melhor resultado.
```

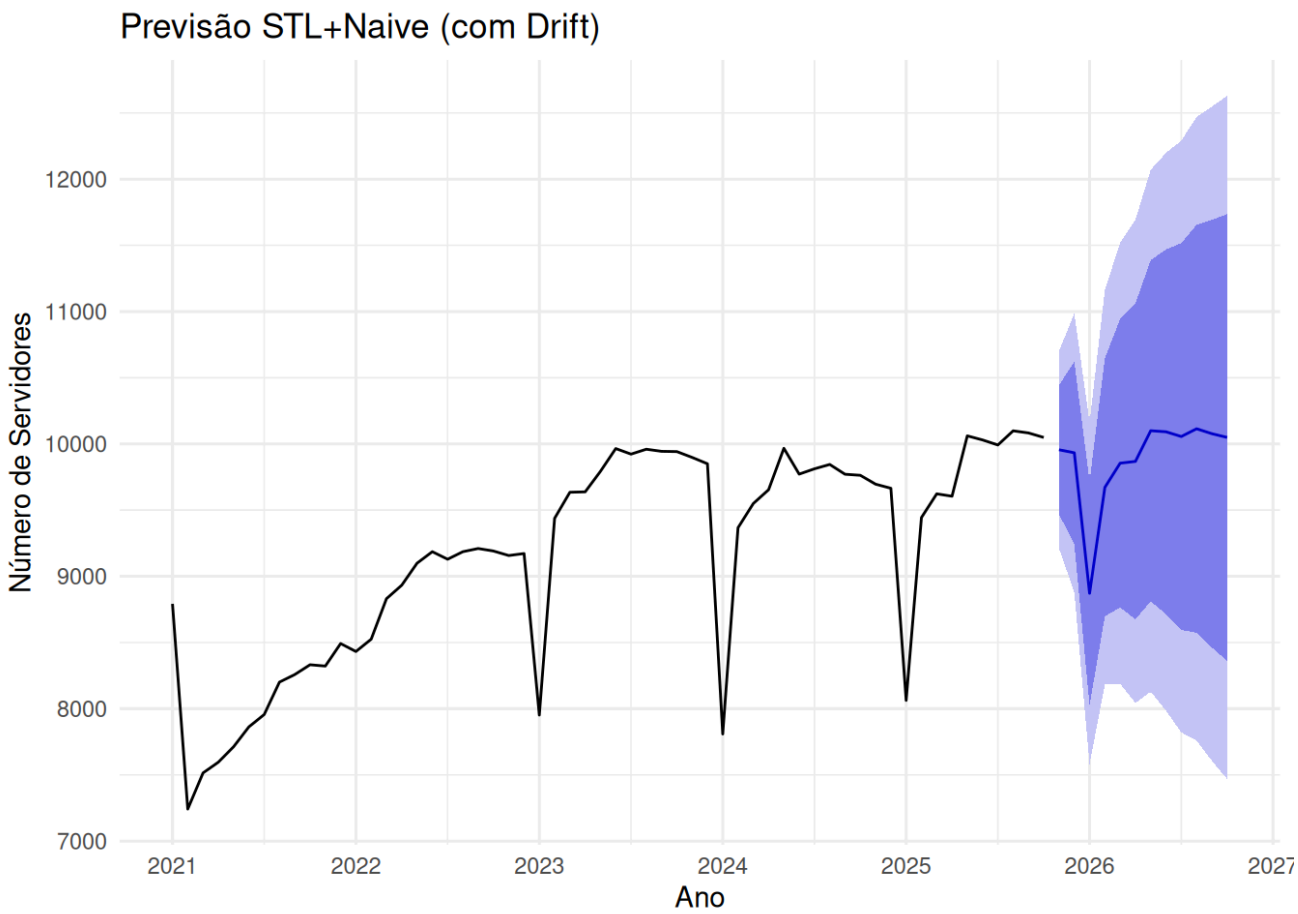
```
#Análise de resíduos do melhor modelo

h <- 12

ultimo_tempo <- as.numeric(tail(time(serie_ts), 1))

tempo_futuro <- seq(from = ultimo_tempo + (1/12),
                    by = 1/12,
                    length.out = h)


ultimo_mes <- as.numeric(tail(cycle(serie_ts), 1))
mes_futuro_num <- rep(1:12, length.out = h + ultimo_mes)[(ultimo_mes + 1):(ultimo_mes +
h)]

novos_dados <- data.frame(
  tempo = tempo_futuro,
  mes_num = mes_futuro_num
)

previsao_stl <- stlf(serie_ts, method = "naive", h = h, level = c(80, 95))
checkresiduals(previsao_stl)
```

```
##
##  Ljung-Box test
##
## data:  Residuals from STL +  Random walk
## Q* = 17.629, df = 12, p-value = 0.1274
##
## Model df: 0.   Total lags used: 12
```

```
cat("p-valor acima de 0.05 que comprova que nossos resíduos são ruído branco (resíduos
independentes), neste modelo não é obrigatório a normalidade dos resíduos, portanto seg
uiremos com o modelo STL+Naive para previsão")
```

```
## p-valor acima de 0.05 que comprova que nossos resíduos são ruído branco (resíduos in
dependentes), neste modelo não é obrigatório a normalidade dos resíduos, portanto segui
remos com o modelo STL+Naive para previsão
```

```
#Previsão STL + Naive



autoplot(previsao_stl) +
  ggtitle("Previsão STL+Naive (com Drift)") +
  xlab("Ano") +
  ylab("Número de Servidores") +
  guides(fill = guide_legend(title = "Intervalo de Confiança")) +
  theme_minimal()
```



```r
#Valores da previsão (Média, Limite Inferior e Superior de 95%)
tabela_previsao <- data.frame(
  Mes_Ano = zoo::as.yearmon(time(previsao_stl$mean)),
  Previsao_Sugerida = as.numeric(previsao_stl$mean),
  Limite_Min_95 = as.numeric(previsao_stl$lower[,2]),
  Limite_Max_95 = as.numeric(previsao_stl$upper[,2])
)

# Arredondando
tabela_previsao[,2:4] <- round(tabela_previsao[,2:4])

print(tabela_previsao)
```

```
##     Mes_Ano Previsao_Sugerida Limite_Min_95 Limite_Max_95
## 1  Nov 2025              9956          9211         10701
## 2  Dec 2025              9933          8880         10987
## 3  Jan 2026              8872          7582         10162
## 4  Feb 2026              9672          8182         11162
## 5  Mar 2026              9855          8189         11520
## 6  Apr 2026              9868          8043         11693
## 7  May 2026             10100          8129         12071
## 8  Jun 2026             10092          7985         12199
## 9  Jul 2026             10056          7821         12291
## 10 Aug 2026             10115          7759         12470
## 11 Sep 2026             10078          7607         12548
## 12 Oct 2026             10049          7469         12629
```

Referências:

Dados: PREFEITURA DE UBERLÂNDIA. Catálogo de Dados Abertos: Servidores Municipais. Disponível em: https://www.uberlandia.mg.gov.br/portal-da-transparencia/dados-abertos/catalogo-de-dados-abertos/ (https://www.uberlandia.mg.gov.br/portal-da-transparencia/dados-abertos/catalogo-de-dados-abertos/). Acesso em: mar. 2026.

Teoria Estatística: HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: Principles and Practice. 3. ed. OTexts: Melbourne, Australia, 2021.

Software R: R CORE TEAM. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria, 2026.

Pacote Forecast: HYNDMAN, R. J.; KHANDAKAR, Y. Automatic time series forecasting: the forecast package for R. Journal of Statistical Software, v. 26, n. 3, p. 1-22, 2008.